

claude-windows / f81fb2b5-e34b-427e-a8a1-a37eb4539a49

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\subagents\workflows\wf\_56274ff0-603\agent-a86fdf72efdb88244.jsonl`
- SHA-256: `041276d07117a7f12540d19cb6f92ed2a2aa4018a12a65dc2336333a2f266390`
- Source modified: `2026-06-21T02:36:28+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf\_56274ff0-603`
- session_id: `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`

Transcript

1. user (2026-06-21T02:34:25.858Z)

Map the INPUT-SOURCE + STORAGE + CONFIG seams that the deployment profile must orchestrate. Read backend/storage/*.py (parse_uri + fetch/put/list_uris for local/gcs/s3/gdrive ? note gdrive is a MOUNTED-drive backend, and the new per-video workspace helper backend/utils/workspace.py + storage.{workspace_root,single_folder_per_video} config), backend/worker/__main__.py (cmd_infer: --video-uri resolution + scratch), backend/config/models.py (StorageConfig, IndexingConfig.detector_impl auto/stub/native, OutputConfig). Identify: how the input source AND the compute backend are selected TODAY, what a single "deployment profile / startup config" would need to add to pick {local-all | cloud-serving} + input {local path / USB / gdrive mount / bucket} + compute {local GPU process | Cloud Run job}, and which knobs already exist vs are new. Tag kind + reuse_status.

2. assistant (2026-06-21T02:34:27.745Z)

I'll map the deployment profile seams by analyzing the storage, config, and worker components. Let me start by exploring the codebase structure.

3. user (2026-06-21T02:34:28.285Z)

```
backend\eval\__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\providers\__init__.py
backend\sports\__init__.py
backend\sports\badminton.py
backend\sports\base.py
backend\utils\__init__.py
backend\utils\geometry.py
backend\annotations\base.py
backend\annotations\file_provider.py
backend\annotations\automated_provider.py
backend\annotations\__init__.py
backend\tools\rally_slicer.py
backend\tools\__init__.py
backend\eval\gemini_refine.py
backend\eval\audio\__init__.py
backend\eval\audio\e1_probe.py
backend\eval\export_wasb_segments.py
backend\pipeline\audio\__init__.py
backend\pipeline\validators\__init__.py
backend\eval\metrics.py
backend\eval\regression.py
backend\eval\annotations.py
backend\features\__init__.py
backend\reporting\harvest.py
```

backend\reporting\run_signals.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\features\rally_seq.py
backend\pipeline\detectors__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
backend\providers\base.py
backend\reporting__init__.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\pipeline\audio\hit_detector.py
backend\pipeline\validators\base.py
backend\utils\paths.py
backend\utils\video.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters__init__.py
backend\tools\export.py
backend\utils\logging_setup.py
backend\pipeline\detectors\audio_features.py
backend\eval\eval_stats.py
backend\eval\score_calibration.py
backend\pipeline\detectors\person_detector.py
backend\infrastructure\execution_policy.py
backend\utils\run_telemetry.py
backend\eval\batch.py
backend\utils\hashing.py
backend\eval\fusion_golden.py
backend\eval\per_user_gate.py
backend\eval\per_user_eval.py
backend\eval\per_user_regression.py
.claude\worktrees\bhi-execpolicy\backend\annotations__init__.py
.claude\worktrees\bhi-execpolicy\backend\annotations\automated_provider.py
.claude\worktrees\bhi-execpolicy\backend\annotations\base.py
.claude\worktrees\bhi-execpolicy\backend\annotations\file_provider.py
.claude\worktrees\bhi-execpolicy\backend\config__init__.py
.claude\worktrees\bhi-execpolicy\backend\config\loader.py
.claude\worktrees\bhi-execpolicy\backend\config\models.py
.claude\worktrees\bhi-execpolicy\backend\eval__init__.py
.claude\worktrees\bhi-execpolicy\backend\eval\ablation.py
.claude\worktrees\bhi-execpolicy\backend\eval\annotations.py
.claude\worktrees\bhi-execpolicy\backend\eval\audio\e1_probe.py
.claude\worktrees\bhi-execpolicy\backend\eval\audio__init__.py
.claude\worktrees\bhi-execpolicy\backend\eval\batch.py
.claude\worktrees\bhi-execpolicy\backend\eval\calibrate_local.py
.claude\worktrees\bhi-execpolicy\backend\eval\calibrate_wasb.py
.claude\worktrees\bhi-execpolicy\backend\eval\calibration.py
.claude\worktrees\bhi-execpolicy\backend\eval\classifier.py
.claude\worktrees\bhi-execpolicy\backend\eval\distill_local.py
.claude\worktrees\bhi-execpolicy\backend\eval\eval_partial_labels.py
.claude\worktrees\bhi-execpolicy\backend\eval\eval_stats.py
.claude\worktrees\bhi-execpolicy\backend\eval\experiment.py
.claude\worktrees\bhi-execpolicy\backend\eval\export_wasb_segments.py
.claude\worktrees\bhi-execpolicy\backend\eval\fusion_audio.py
.claude\worktrees\bhi-execpolicy\backend\eval\fusion_compare.py

```
.claude\worktrees\bhi-execpolicy\backend\eval\fusion_golden.py
.claude\worktrees\bhi-execpolicy\backend\eval\gemini_refine.py
.claude\worktrees\bhi-execpolicy\backend\eval\gt_loader.py
.claude\worktrees\bhi-execpolicy\backend\eval\harness.py
.claude\worktrees\bhi-execpolicy\backend\eval\metrics.py
.claude\worktrees\bhi-execpolicy\backend\eval\per_user_eval.py
.claude\worktrees\bhi-execpolicy\backend\eval\per_user_gate.py
.claude\worktrees\bhi-execpolicy\backend\eval\per_user_regression.py
.claude\worktrees\bhi-execpolicy\backend\eval\rally_seq_proto.py
(Results are truncated. Consider using a more specific path or pattern.)
```

4. user (2026-06-21T02:34:28.791Z)

```
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/annotations/automated_provider.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/annotations/base.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/annotations/file_provider.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/annotations/__init__.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/loader.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/__init__.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/ablation.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/annotations.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/audio/e1_probe.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/audio/__init__.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/batch.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibrate_local.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibrate_wasb.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibration.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/classifier.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/distill_local.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/eval_partial_labels.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/eval_stats.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/experiment.py
```

5. assistant (2026-06-21T02:34:32.066Z)

Now let me read the key files mentioned in your task:

6. user (2026-06-21T02:34:32.778Z)

```
EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\storage'
```

7. user (2026-06-21T02:34:33.132Z)

Exit code 2

/usr/bin/bash: eval: line 1: unexpected EOF while looking for matching `''

8. user (2026-06-21T02:34:35.113Z)

```
backend\storage\gcs.py
backend\storage\local.py
backend\storage\s3.py
backend\storage\__init__.py
backend\storage\base.py
backend\storage\ref.py
backend\storage\gdrive.py
```

9. user (2026-06-21T02:34:35.387Z)

```
backend\utils\workspace.py
```

10. user (2026-06-21T02:34:35.509Z)

```
backend\worker\__main__.py
```

11. user (2026-06-21T02:34:35.593Z)

```
backend\config\models.py
```

12. assistant (2026-06-21T02:34:37.424Z)

Now I'll read the critical files to understand the deployment profile seams:

13. user (2026-06-21T02:34:37.655Z)

```
1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      The models support graceful migration from old config.json files via extra="allow".
7      """
8
9      from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
```

```

33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         ---
72         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
73         device: str = "cuda" # torch device for the native runner: cuda | mps | cpu
74         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
75         `conda activate`)
76         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
78         regenerable
79         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
80         resumes.
81         keep_frames: bool = False
82         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
83         disabled.
84         min_free_gb: float = 0.0
85         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
86         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
87         for a
88         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
89         to the
90         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
91         WSL disk,
92         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
93         deterministic).
94         # Tri-state:
95         # None = per-runner default ? the WSL runner keeps DISK extraction
96         (unchanged Windows

```

```

87         #             behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
88         #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
89         stream_video: Optional[bool] = None
90
91
92     class FusionConfig(BaseModel):
93         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
94
95         Every default reproduces control behaviour: the fusion segmenter persists the
96         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
97         true ? the person-cue features, but it does NOT gate the served handoff unless a
98         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
99         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
100        supplied ? off-court persons (spectators / adjacent courts) are excluded.
101        """
102        # Which trajectory substrate seeds the windows (shuttle stays primary).
103        substrate: Literal["wasb", "tracknet"] = "wasb"
104        # Windowing velocity threshold for the fusion family (the engine reads
105        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
106        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
107        velocity_threshold: float = 0.02
108        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
109        # enabled ? so the default path never imports torch / touches the GPU.
110        cue_flags: dict[str, bool] = Field(default_factory=dict)
111        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
112        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
113        min_crossings: int = Field(default=0, ge=0)
114        # Served Gate B: when the person cue is on, drop windows with median in-court
players
115        # < this. 0 = OFF.
116        min_players: int = Field(default=0, ge=0)
117        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
118        court_polygon: Optional[list[list[float]]] = None
119        # Net line for the person two-sided-motion split (person features only ? the shuttle
120        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
121        net_axis: Literal["x", "y"] = "y"
122        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
123
124
125     class IndexingConfig(BaseModel):
126         default_segmenter: str = "yolo_hybrid"
127         use_gpu: bool = True
128         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
129         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
130         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
131         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
132         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
133         detector_impl: str = "auto"
134         motion_threshold: float = 0.08
135         min_segment_duration: float = 3.0
136         dead_time_merge_gap: float = 2.0
137         chunk_duration: float = 90.0
138         chunk_overlap: float = 10.0
139         detector_model: str = "fasterrcnn_resnet50_fpn"
140         detector_score_threshold: float = 0.5
141         yolo_velocity_threshold: float = 0.15

```

```

142     yolo_frame_skip: int = 3
143     yolo_tracker: str = "bytetrack.yaml"
144     yolo_prefetch_workers: int = 2
145     min_action_window_duration: float = 2.0
146     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
147     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
148     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
149     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
150     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
151     # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
152     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
153     max_window_duration: float = 6.0
154     window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
155     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
156     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
157     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
158     # Only cuts (recall can't drop). OFF by default until measured/promoted.
159     window_hard_cap: bool = False
160     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
161     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
162     # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
163     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
164     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
165     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
166     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
167     window_inactivity_end: float = 1.5
168     inactivity_rally_thresh: float = 0.20
169     inactivity_min_density: float = 0.30
170     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
171     # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
172     # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
173     # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
174     # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
175     # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
176     # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
177     # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
178     window_blob_fallback: bool = False
179     # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
180     # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual

```

```

is ~11x).
181     # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
182     # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
183     window_blob_factor: float = 4.0
184     log_yolo_candidates: bool = True
185     store_yolo_candidates: bool = True
186     ai_handoff_padding: float = 2.0
187     ai_max_workers: int = 5
188     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
189     # chunks of a high-bitrate (4K) source blow past the provider upload cap
190     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
191     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
192     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
193     # Matches gemini_refine's --clip-scale-width default.
194     ai_chunk_scale_width: int = 1280
195     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
196     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
197     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
198     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
199     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
200     skip_ai_handoff: bool = False
201     # Optional debug knob: trim every video to the first N seconds before processing.
202     debug_duration: Optional[float] = None
203
204     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
205     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
206     fusion: FusionConfig = Field(default_factory=FusionConfig)
207
208     @field_validator("default_segmenter")
209     @classmethod
210     def segmenter_must_be_registered(cls, v: str) -> str:
211         # Registry check happens at runtime in main/segmenter selection.
212         # Here we just allow any string for forward compatibility.
213         return v
214
215     @model_validator(mode="after")
216     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
217         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
218         # step makes `while t < duration: t += step` loop forever, growing memory before
any
219         # GPU work. Reject the bad config at load time instead.
220         if self.chunk_overlap >= self.chunk_duration:
221             raise ValueError(
222                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
223                 f"({self.chunk_duration}); otherwise chunking never advances."
224             )
225         return self
226
227
228     class OutputConfig(BaseModel):
229         default_highlights_name: str = "highlights.mp4"
230         db_name: str = "sports_indexer.db"
231         # Directory where the DB, highlight reels, and processed clips are written.
232         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
233         output_dir: str = "output"
234
235

```



```

236     class WatermarkConfig(BaseModel):
237         """Branding overlay burned into each highlight clip during the per-clip re-encode
238         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
239         (under `stitching.watermark` in config.json) to turn it off. The text is fully
240         configurable. The concat stage stays stream-copy (-c copy)."""
241         enabled: bool = True
242         text: str = "Generated by Khelsutra.guru"
243         logo_path: str = ""          # optional transparent PNG overlay; "" = no logo
244         font_path: str = ""          # optional .ttf; "" = use the fontconfig `font` family
below
245         font: str = "Sans"           # fontconfig family used when font_path is empty
246         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
247         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
248         box: bool = True             # faint backing box behind the text for legibility
249         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
250         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
251
252
253     class StitchingConfig(BaseModel):
254         begin_padding: float = 0.5
255         end_padding: float = 0.5
256         use_cache: bool = False
257         min_shot_count: int = 1
258         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
259         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
260         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
261         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
262         # local detector over-fires and its false positives are mostly SHORT (motion blips,
263         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
264         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
265         min_rally_duration: float = Field(default=0.0, ge=0.0)
266         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
267
268
269     class LoggingConfig(BaseModel):
270         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
271         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
272         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
273         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
274         # them to WARNING; set true to restore full chatter for request-flow debugging.
275         verbose_http: bool = False
276
277
278     class SecurityConfig(BaseModel):
279         # When set, /api/process video_path must resolve under this directory (hosted/tenant
280         # mode). Default None preserves the single-user local 'any local file' workflow.
281         allowed_video_root: Optional[str] = None
282
283         # --- Chunked upload (B2, PR-2) ---
284         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
285         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
286         # contain upload_dir or /api/process will reject the uploaded paths.
287         upload_dir: str = "output/uploads"
288         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
289         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
290         max_upload_mb: int = 4096
291         max_part_mb: int = 95
292         allowed_upload_extensions: List[str] = ["mp4", "mov"]

```

```

293
294
295     class StorageConfig(BaseModel):
296         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
297         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
298         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
299         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
300         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
301         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
302         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
303         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
304         output_root: str = ""
305         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
306         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
307         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
308         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
309         # the legacy flat layout is used unchanged.
310         single_folder_per_video: bool = False
311         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
312         workspace_root: str = ""
313         # Cloud namespace prefix under the root so per-video folders sit tidily beside
314         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
315         workspace_prefix: str = "workspaces"
316         local: dict[str, Any] = Field(default_factory=dict)
317         s3: dict[str, Any] = Field(default_factory=dict)
318         gcs: dict[str, Any] = Field(default_factory=dict)
319         gdrive: dict[str, Any] = Field(default_factory=dict)
320
321
322     class AppConfig(BaseModel):
323         """Root configuration object.
324
325         All runtime configuration should be accessed via an instance of this class
326         (or the loader) rather than raw dict lookups.
327         """
328
329         sport: Sport = "badminton"
330         ai_provider: str = "gemini"
331         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
332         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
333         ai_model: str = "gemini-flash-latest"
334
335         validation: ValidationConfig = Field(default_factory=ValidationConfig)
336         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
337         output: OutputConfig = Field(default_factory=OutputConfig)
338         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
339         security: SecurityConfig = Field(default_factory=SecurityConfig)
340         logging: LoggingConfig = Field(default_factory=LoggingConfig)
341         storage: StorageConfig = Field(default_factory=StorageConfig)
342
343         # Allow extra keys during transition so existing config.json files do not break.
344         model_config = {
345             "extra": "allow",
346             "validate_assignment": True,

```

```

347     }
348
349     # Temporary migration shim for legacy dict-style access (config.get("section",
350     {}).get(...)).
351     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
352     # attribute access / the typed config. A nested *section* is returned as a plain
353     dict
354     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
355     # section model instead breaks it (a model has no .get), which bit every validator
356     on
357     # real runs. Scalar leaves are returned as-is.
358     def get(self, key: str, default: Any = None) -> Any:
359         _MISSING = object()
360         val: Any = _MISSING
361         if key in type(self).model_fields or hasattr(self, key):
362             val = getattr(self, key, _MISSING)
363         if val is _MISSING:
364             # Legacy convenience: find a leaf key inside any nested section. Sections
365             # are
366             # discovered dynamically from the model fields, so a future top-level
367             # section
368             # (e.g. `audio`, `jobs`) is covered automatically without editing a
369             # hardcoded list.
370             for fname in type(self).model_fields:
371                 section = getattr(self, fname, None)
372                 if isinstance(section, BaseModel) and hasattr(section, key):
373                     val = getattr(section, key)
374                     break
375             if val is _MISSING:
376                 return default
377             if isinstance(val, BaseModel):
378                 return val.model_dump(mode="json")
379             return val
380
381     def __getitem__(self, key: str) -> Any:
382         val = self.get(key)
383         if val is None and key not in type(self).model_fields:
384             # Re-raise like a real dict for unknown top-level
385             raise KeyError(key)
386         return val
387
388     # Convenience type alias for callers that want to annotate
389     Config = AppConfig
390
391

```

14. user (2026-06-21T02:34:37.882Z)

```

1     """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3     Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
4     backends
5     live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
6     never
7     pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
8     Drive ?
9     no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
10    """
11    from __future__ import annotations
12
13    import os
14    from abc import ABC, abstractmethod
15    from typing import Iterator
16
17    from backend.storage.ref import StorageRef, StorageStat

```

```

15
16
17 class StorageBackend(ABC):
18     """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19     worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21     @abstractmethod
22     def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                       *, overwrite: bool = False) -> str:
24         """Make the object available as a local file; return its local path. For the
25         ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27     @abstractmethod
28     def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29            content_type: "str | None" = None) -> StorageRef:
30         """Upload/copy a local file to ``ref``; return ``ref``."""
31
32     @abstractmethod
33     def exists(self, ref: StorageRef) -> bool: ...
34
35     @abstractmethod
36     def stat(self, ref: StorageRef) -> StorageStat: ...
37
38     @abstractmethod
39     def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40         """Yield a ``StorageRef`` per object under ``prefix``."""
41
42     @abstractmethod
43     def delete(self, ref: StorageRef) -> None: ...
44
45     def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
46         """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47         sign (``local`` cannot; Drive only emulates) ? callers must treat
48         ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49         raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50

```

15. user (2026-06-21T02:34:37.964Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    """
16    from __future__ import annotations
17
18    import argparse
19    import csv

```

```

20     import json
21     import os
22     import sys
23     import tempfile
24     from typing import Any, List
25
26     from backend import storage
27     from backend.config import load_config
28     from backend.infrastructure.database import Database
29     from backend.pipeline.segmenters.base import segmenter_registry
30     from backend.pipeline.validators.base import registry as validator_registry
31     from backend.reporting import emit_indexer_report
32     from backend.results import Failure
33     from backend.utils.hashing import compute_video_id
34
35
36     def _coerce(v: str) -> Any:
37         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
38         str."""
39         low = v.lower()
40         if low in ("true", "false"):
41             return low == "true"
42         for cast in (int, float):
43             try:
44                 return cast(v)
45             except ValueError:
46                 pass
47         return v
48
49     def _apply_overrides(config: Any, sets: List[str]) -> None:
50         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
51         indexing.skip_ai_handoff=true)."""
52         for kv in sets or []:
53             key, sep, val = kv.partition("=")
54             if not sep:
55                 raise ValueError(f"--set expects k=v, got {kv!r}")
56             parts = key.split(".")
57             obj = config
58             for p in parts[:-1]:
59                 obj = getattr(obj, p)
60             setattr(obj, parts[-1], _coerce(val))
61
62     def _write_intervals_csv(segments: List[dict], path: str) -> None:
63         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
64         friendly
65         artifact (same schema as the rally-annotator labels)."""
66         with open(path, "w", newline="") as f:
67             w = csv.writer(f)
68             w.writerow(["start", "end", "shot_count", "ending_reason"])
69             for s in segments:
70                 w.writerow([
71                     f'{float(s.get("start_time", 0.0)):.3f}',
72                     f'{float(s.get("end_time", 0.0)):.3f}',
73                     s.get("shot_count", ""),
74                     s.get("ending_reason", s.get("shot_count_confidence", "")),
75                 ])
76
77     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
78     None:
79         with open(path, "w") as f:
80             json.dump({
81                 "video_id": video_id,

```

```

81         "status": status,
82         "segment_count": len(segments),
83         "segments": [{"start": float(s.get("start_time", 0.0)),
84                       "end": float(s.get("end_time", 0.0))} for s in segments],
85     }, f, indent=2)
86
87
88     def cmd_infer(args: argparse.Namespace) -> int:
89         config = load_config(args.config) if args.config else load_config()
90         _apply_overrides(config, args.set)
91         storage_cfg = config.storage.model_dump()
92
93         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
94         os.makedirs(scratch, exist_ok=True)
95         config.output.output_dir = scratch
96
97         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
98         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
99         config=storage_cfg)
100         print(f"[worker] pulled {args.video_uri} -> {local_video}")
101
102         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
103         byte-identical).
104         video_id = compute_video_id(local_video)
105
106         # 3. VALIDATE (same registry as the main CLI).
107         val_results, val_context = validator_registry.run_all_with_context(local_video,
108         config)
109         failures = [r for r in val_results if not r.passed]
110         db = Database(os.path.join(scratch, config.output.db_name))
111         db.add_video(video_id, local_video, "failed" if failures else "processed",
112         [r.model_dump() for r in val_results])
113
114         segments: List[dict] = []
115         segmenter_actual = None
116         segmentation_ran = False
117         status = "failed"
118         try:
119             if failures:
120                 print("[worker] validation FAILED: "
121                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
122                 return 2
123             seg_name = args.segmenter or config.indexing.default_segmenter
124             segmenter_cls = segmenter_registry.get(seg_name)
125             if not segmenter_cls:
126                 print(f"[worker] unknown segmenter: {seg_name!r} "
127                 f"(have {list(segmenter_registry.list_available())})")
128                 return 2
129             segmenter_actual = seg_name
130             result = segmenter_cls(db, config).process_video(video_id, local_video,
131             max_frames=args.max_frames)
132             segmentation_ran = True
133             if isinstance(result, Failure):
134                 print(f"[worker] segmenter FAILED: {result.message}")
135                 return 3
136             segments = result.value if hasattr(result, "value") else result
137             status = "success"
138         finally:
139             # Best-effort per-video observability report (never raises) ? parity with the
140             main CLI.
141             emit_indexer_report(
142                 db=db, video_id=video_id, video_path=local_video, config=config,
143                 val_results=val_results, val_context=val_context,
144                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,

```

```

140         was_reingest=False, segmentation_ran=segmentation_ran,
141     )
142
143     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
144     out_local = os.path.join(scratch, "outputs", video_id)
145     os.makedirs(out_local, exist_ok=True)
146     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
147     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
148
149     out_prefix = args.out_uri.rstrip("/")
150     pushed = []
151     for fn in sorted(os.listdir(out_local)):
152         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
154         pushed.append(uri)
155
156     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
157     for u in pushed:
158         print("    ", u)
159     return 0
160
161
162     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
163     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
164     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
165
166
167     def _probe_video_fps_width(path: str):
168         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
169
170         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
171         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
172         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
173         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
174         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
175         rather than guesses.
176         """
177         import json
178         import subprocess
179         try:
180             out = subprocess.check_output(
181                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
182                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
183                 stderr=subprocess.DEVNULL).decode()
184             st = (json.loads(out).get("streams") or [{}])[0]
185             num, den = str(st.get("r_frame_rate", "0/1")).partition("/")
186             fps = float(num) / float(den or "1")
187             width = float(st.get("width") or 0)
188             if fps > 0 and width > 0:
189                 return fps, width
190         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
191             pass
192         return None
193
194

```

```

195 def _resolve_train_detector(args: argparse.Namespace, config: Any):
196     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
197     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
198     stub=CPU mock). Returns the runner, or prints an error + returns None.
199
200     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
201     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
202     no shuttle detector and is rejected.
203     """
204     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
205
206     seg_cls = segmenter_registry.get(args.segmenter)
207     if not seg_cls:
208         print(f"[worker] unknown segmenter: {args.segmenter!r} "
209               f"(have {list(segmenter_registry.list_available())})")
210         return None
211     seg = seg_cls(None, config)
212     if not isinstance(seg, TrajectoryHybridSegmenter):
213         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
214               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
215         return None
216     try:
217         runner, _ = seg._resolve_runner(config.get("indexing", {}))
218     except NotImplementedError as e:
219         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
220         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
221         print(f"[worker] detector not available: {e}")
222         return None
223     ok, msg = runner.healthcheck()
224     if not ok:
225         print(f"[worker] detector environment not ready: {msg}")
226         return None
227     print(f"[worker] detector ready ({args.segmenter}): {msg}")
228     return runner
229
230
231 def cmd_train(args: argparse.Namespace) -> int:
232     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
233     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
234
235     Two trajectory sources (the train pipeline + harness are identical either way):
236     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
237         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
238     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
239         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
240         is the M3.5 "first real training" path; only the trajectory source flips.
241     """
242     import subprocess
243
244     config = load_config(args.config) if args.config else load_config()
245     _apply_overrides(config, args.set)
246     storage_cfg = config.storage.model_dump()
247
248     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
249     labels_dir = os.path.join(scratch, "labels")

```



```

250     traj_dir = os.path.join(scratch, "trajectories")
251     videos_dir = os.path.join(scratch, "videos")
252     os.makedirs(labels_dir, exist_ok=True)
253     os.makedirs(traj_dir, exist_ok=True)
254
255     corpus = args.corpus_uri.rstrip("/")
256     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
257                         if u.lower().endswith(".csv"))
258     if len(label_uris) < 2:
259         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
260             f"under {corpus}/labels/")
261     return 2
262
263     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
264     runner = None
265     video_by_stem: dict = {}
266     if args.trajectory_source == "detector":
267         os.makedirs(videos_dir, exist_ok=True)
268         runner = _resolve_train_detector(args, config)
269         if runner is None:
270             return 2
271         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
272             vname = storage.parse_uri(vuri).name
273             if not vname.lower().endswith(_VIDEO_EXTS):
274                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
275             video_by_stem[os.path.splitext(vname)[0]] = vuri
276
277     print(f"[worker] trajectory source: {args.trajectory_source}")
278     specs: List[dict] = []
279     for uri in label_uris:
280         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
281         name = fname.replace(".rallies.csv", "").replace(".csv", "")
282         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
283         if args.trajectory_source == "detector":
284             vuri = video_by_stem.get(name)
285             if not vuri:
286                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
287                 continue
288             local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
289                                     config=storage_cfg)
290             video_id = compute_video_id(local_video)
291             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
292             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
293             meta = _probe_video_fps_width(local_video)
294             if meta is None:
295                 print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
296                 continue
297             fps, frame_width = meta
298             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
299             if not traj:
300                 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
301                 continue
302             specs.append({"name": name, "trajectory": traj, "labels": local_label,

```

```

303             "fps": fps, "frame_width": frame_width})
304         else:
305             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
306             traj = os.path.join(traj_dir, f"{name}_traj.csv")
307             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
308             specs.append({"name": name, "trajectory": traj, "labels": local_label,
309                           "fps": args.fps, "frame_width": args.frame_width})
310
311         if len(specs) < 2:
312             print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
313             return 2
314         manifest_path = os.path.join(scratch, "manifest.json")
315         with open(manifest_path, "w", encoding="utf-8") as f:
316             json.dump(specs, f, indent=2)
317         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
318         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
319
320         out_dir = os.path.join(scratch, "artifacts")
321         cmd = [sys.executable, "-m", "training.gen0.harness",
322               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
323         if args.floor:
324             floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
325                            if "://" in args.floor else args.floor)
326             cmd += ["--floor", floor_local]
327         print("[worker] training:", " ".join(cmd))
328         rc = subprocess.run(cmd).returncode
329         if rc != 0:
330             print(f"[worker] gen0 harness failed (rc={rc})")
331             return rc
332
333         # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
334         bundle = os.path.join(out_dir, args.version)
335         out_prefix = args.out_uri.rstrip("/")
336         pushed = []
337         for fn in sorted(os.listdir(bundle)):
338             uri = f"{out_prefix}/weights/{args.version}/{fn}"
339             storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
340             pushed.append(uri)
341         print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
342         for u in pushed:
343             print("    ", u)
344         return 0
345
346
347     def build_parser() -> argparse.ArgumentParser:
348         p = argparse.ArgumentParser(prog="python -m backend.worker",
349                                     description="Storage-aware worker: pull ? run pipeline ?
push.")
350         sub = p.add_subparsers(dest="cmd", required=True)
351
352         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
353         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
354         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
355         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
356         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
357         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp

```

```

dir)")
358     pi.add_argument("--max-frames", type=int, default=None)
359     pi.add_argument("--set", action="append", default=[], metavar="k=v",
360                     help="config override, e.g. --set indexing.skip_ai_handoff=true")
361     pi.set_defaults(func=cmd_infer)
362
363     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
364     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
365                     "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
366     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
367     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
368     pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
369                     help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
370                     "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
371                     "(needs videos/ in the corpus; honours detector_impl).")
372     pt.add_argument("--segmenter", default="wasb_hybrid",
373                     help="trajectory hybrid whose detector is used for --trajectory-
source detector")
374     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
375     pt.add_argument("--config", default=None)
376     pt.add_argument("--scratch", default=None)
377     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
378     pt.add_argument("--frame-width", type=float, default=1280.0,
379                     help="synth trajectory frame width (detector probes the video)")
380     pt.add_argument("--set", action="append", default=[], metavar="k=v")
381     pt.set_defaults(func=cmd_train)
382     return p
383
384
385     def main(argv=None) -> int:
386         args = build_parser().parse_args(argv)
387         return args.func(args)
388
389
390     if __name__ == "__main__":
391         sys.exit(main())
392

```

16. assistant (2026-06-21T02:34:40.359Z)

Now let me read the storage implementations and the workspace utility:

17. user (2026-06-21T02:34:40.871Z)

```

1     """Per-video workspace path helper ? one folder per video, local?cloud identical.
2
3     See ``docs/PER_VIDEO_WORKSPACE.md``. The goal: every per-video artifact lives under
4     ONE folder keyed by the MD5 ``video_id``
5     (``backend/utls/hashing.py::compute_video_id``):
6
7     <root>/[<prefix>]/<video_id>/{source,detector,outputs,reels,logs,tmp}/...
8
9     The ``root`` is a **storage URI** (``local`` path / ``gcs://`` / ``s3://`` /
``gdrive://``), so a
10    LOCAL deployment and a CLOUD (bucket) deployment share the *same relative layout* ? only
the root
differs. This module is **pure** (string/URI joins, no I/O) so it is trivially testable

```

```

and works
11     for every backend; the storage layer (`backend/storage/`) does the actual read/write.
12
13     Two roots by design (D3): a **durable** workspace root (may be `gcs://`) for artifacts
worth
14     keeping, and a **fast local compute-scratch** mirror (always local) for frame caches /
extracted
15     chunks that must never be written to a cloud root. `tmp/` is the local-only bucket.
16
17     This is Phase 0: the helper + config fields only ? NOT yet wired into write sites (that
converges
18     in P2?P4, flag-gated by `storage.single_folder_per_video`). No behaviour changes on
import.
19     """
20     from __future__ import annotations
21
22     import os
23     from dataclasses import dataclass
24
25     # Per-video subfolders. `tmp` is LOCAL-ONLY (frame PNGs, handoff/yolo/gemini chunks,
compile
26     # clips) ? huge + ephemeral, never pushed to a cloud workspace; cache-hygiene drops it.
27     SUBDIRS = ("source", "detector", "outputs", "reels", "logs", "tmp")
28     LOCAL_ONLY_SUBDIRS = ("tmp",)
29
30
31     def join_uri(root: str, *parts: str) -> str:
32         """Join path components onto a storage root with forward slashes.
33
34         Matches how the codebase already composes storage URIs (e.g.
`f"{prefix}/weights/..."` in
35         `backend/worker`). Works for `gcs://bucket`, `s3://bucket`, `local:///abs`
and bare
36         paths alike; empty/blank parts are skipped so `join_uri(root, "", "x")` ==
`join_uri(root,
37         "x")`. Forward slashes are valid path separators in Python on Windows too.
38         """
39         base = root.rstrip("/")
40         # Skip empty / whitespace-only parts; for real parts strip only slashes so spaces
inside a
41         # name (e.g. a mounted "My Drive" path) are preserved.
42         tail = "/" .join(p.strip("/") for p in parts if p and p.strip())
43         return f"{base}/{tail}" if tail else base
44
45
46     def resolve_workspace_root(workspace_root: str, output_root: str, output_dir: str) ->
str:
47         """Durable workspace root, by precedence: `storage.workspace_root` ?
48         `storage.output_root` ? `output.output_dir`. The first non-empty wins (non-
breaking)."""
49         return (workspace_root or "").strip() or (output_root or "").strip() or (output_dir
or "output")
50
51
52     @dataclass(frozen=True)
53     class VideoWorkspace:
54         """The single folder for one video. Pure path math; build with :func:`for_video`.
55
56         `base` = `

```

```

62
63     @property
64     def base(self) -> str:
65         if self.prefix:
66             return join_uri(self.root, self.prefix, self.video_id)
67             return join_uri(self.root, self.video_id)
68
69     def path(self, sub: str = "", name: str = "") -> str:
70         """A path under this video's folder: ``base[/sub][/name]``."""
71         return join_uri(self.base, sub, name)
72
73     @property
74     def source(self) -> str:
75         return self.path("source")
76
77     @property
78     def detector(self) -> str:
79         return self.path("detector")
80
81     @property
82     def outputs(self) -> str:
83         return self.path("outputs")
84
85     @property
86     def reels(self) -> str:
87         return self.path("reels")
88
89     @property
90     def logs(self) -> str:
91         return self.path("logs")
92
93     @property
94     def tmp(self) -> str:
95         return self.path("tmp")
96
97
98     def for_video(root: str, video_id: str, prefix: str = "workspaces") -> Videoworkspace:
99         """Build the :class:`Videoworkspace` for ``video_id`` under ``root`` (a storage
100         URI)."""
101         if not video_id:
102             raise ValueError("video_id is required to build a per-video workspace")
103         return Videoworkspace(root=root, video_id=video_id, prefix=prefix)
104
105     def local_scratch_base(scratch_root: str, video_id: str) -> str:
106         """The fast LOCAL compute-scratch mirror for a video (always native FS, never a
107         cloud root).
108         Used for frame caches / extracted chunks (the ``tmp/`` bucket) during compute,
109         before durable
110         artifacts are pushed to the workspace. Uses ``os.path.join`` (OS-native separators)
111         since this
112         is always a local filesystem path.
113         """
114         if not video_id:
115             raise ValueError("video_id is required for a local scratch path")
116         return os.path.join(scratch_root or os.getcwd(), video_id, "tmp")

```

18. user (2026-06-21T02:34:41.069Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
4     imported

```

```

4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive`` likewise needs none ? it reads a locally
MOUNTED
7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE``.
12    """
13    from __future__ import annotations
14
15    import os
16    from typing import Any, Dict, Iterator, Optional
17
18    from backend.storage.base import StorageBackend
19    from backend.storage.ref import StorageRef, StorageStat, parse_uri
20
21    __all__ = [
22        "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
23        "get_storage", "fetch", "put", "list_uris",
24    ]
25
26
27    def _backend_class(name: str):
28        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
29        if name == "local":
30            from backend.storage.local import LocalStorage
31            return LocalStorage
32        if name == "s3":
33            from backend.storage.s3 import S3Storage
34            return S3Storage
35        if name == "gcs":
36            from backend.storage.gcs import GCSStorage
37            return GCSStorage
38        if name == "gdrive":
39            from backend.storage.gdrive import GDriveStorage
40            return GDriveStorage
41        raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
42
43
44    def get_storage(backend: Optional[str] = None, *,
45                    config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
46        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
47        the implementation; its per-backend settings come from ``config[backend]`` merged
with
48        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
49        cfg = config or {}
50        name = backend or cfg.get("backend", "local")
51        settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
52        settings.update(overrides)
53        return _backend_class(name)(**settings)
54
55
56    def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
57              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
58        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
59        (no copy); for s3/gcs it downloads to ``dst``."""
60        ref = parse_uri(uri)

```

```

61         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
62                               overwrite=overwrite)
63
64     def put(local_path: "str | os.PathLike[str]", uri: str, *,
65            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
66     str:
67         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
68         ref = parse_uri(uri)
69         get_storage(ref.backend, config=config).put(local_path, ref,
70             content_type=content_type)
71         return uri
72
73     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
74     Iterator[str]:
75         """Yield the URI of every object under ``prefix_uri``."""
76         ref = parse_uri(prefix_uri)
77         for r in get_storage(ref.backend, config=config).list(ref):
78             yield r.uri

```

19. user (2026-06-21T02:34:41.276Z)

```

1     """Local-filesystem storage backend ? the default, zero new dependencies.
2
3     Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
4     (the
5     file is already local), ``put`` is a copy (a no-op when src == dst). With
6     ``backend='local'``
7     the whole pipeline behaves exactly as before the storage layer existed.
8     """
9     from __future__ import annotations
10
11     import os
12     import shutil
13     from datetime import datetime
14     from typing import Iterator
15
16     from backend.storage.base import StorageBackend
17     from backend.storage.ref import StorageRef, StorageStat
18
19     class LocalStorage(StorageBackend):
20         def __init__(self, *, base_dir: str = "", **_ignored) -> None:
21             # Relative keys resolve under base_dir when set; absolute keys are used as-is.
22             self.base_dir = base_dir
23
24         def _path(self, ref: StorageRef) -> str:
25             p = ref.key
26             if self.base_dir and not os.path.isabs(p):
27                 return os.path.join(self.base_dir, p)
28             return p
29
30         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31             # The object is already on the local FS ? return its path (identity). ``dst`` is
32             # honoured only if it names a *different* location (a copy is then made).
33             src = self._path(ref)
34             if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
35                 return src
36             dst = str(dst)
37             if os.path.exists(dst) and not overwrite:
38                 return dst
39             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
40             shutil.copyfile(src, dst)

```

```

40         return dst
41
42     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43         dst = self._path(ref)
44         if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46             shutil.copyfile(str(local_path), dst)
47         return ref
48
49     def exists(self, ref) -> bool:
50         return os.path.exists(self._path(ref))
51
52     def stat(self, ref) -> StorageStat:
53         st = os.stat(self._path(ref))
54         return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))
55
56     def list(self, prefix) -> Iterator[StorageRef]:
57         base = self._path(prefix)
58         if os.path.isdir(base):
59             for root, _dirs, files in os.walk(base):
60                 for fn in sorted(files):
61                     full = os.path.join(root, fn)
62                     yield StorageRef("local", "", full, f"local://{full}")
63         elif os.path.exists(base):
64             yield StorageRef("local", "", base, f"local://{base}")
65
66     def delete(self, ref) -> None:
67         path = self._path(ref)
68         if os.path.exists(path):
69             os.remove(path)
70

```

20. user (2026-06-21T02:34:41.447Z)

```

1     """Storage value objects + URI parsing for the pluggable storage layer.
2
3     A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5         <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6         local:///<path>            -> local    (local:///abs or local://rel/p)
7         s3:///<bucket>/<key>       -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8         gcs:///<bucket>/<key>     -> gcs      (gs:// accepted, normalized to gcs://)
9         gdrive:///<path>          -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11     No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12     stays cheap.
13     """
14     from __future__ import annotations
15
16     import re
17     from dataclasses import dataclass
18     from datetime import datetime
19     from typing import Optional
20
21     _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22     _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25     @dataclass(frozen=True)
26     class StorageStat:

```



```

27         """Lightweight object metadata (uniform across backends)."""
28         size: int
29         modified: Optional[datetime] = None
30         etag: Optional[str] = None
31         content_type: Optional[str] = None
32
33
34     @dataclass(frozen=True)
35     class StorageRef:
36         """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
37         ``gdrive`` it is
38         empty and ``key`` carries the path (a filesystem path, or a path under the mounted
39         Drive)."""
40         backend: str
41         bucket: str
42         key: str
43         uri: str
44
45         @property
46         def name(self) -> str:
47             return self.key.rstrip("/").rsplit("/", 1)[-1]
48
49     def parse_uri(uri: str) -> StorageRef:
50         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
51
52         A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
53         have no ``//``) is treated as a local filesystem path ? preserving today's
54         behaviour.
55         """
56         s = str(uri)
57         m = _SCHEME_RE.match(s)
58         if not m:
59             return StorageRef("local", "", s, f"local://{s}")
60         scheme = m.group(1).lower()
61         rest = s[m.end():]
62         if scheme == "gs":
63             scheme = "gcs"
64         if scheme in ("local", "gdrive"):
65             # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
66             Drive".
67             return StorageRef(scheme, "", rest, s)
68         if scheme in _KNOWN:
69             bucket, _, key = rest.partition("/")
70             return StorageRef(scheme, bucket, key, s)
71         raise ValueError(f"unsupported storage URI scheme: {scheme}://{rest} (in {s})")

```

21. assistant (2026-06-21T02:34:43.489Z)

Now let me search for how input source selection and compute backend selection happen today:

22. user (2026-06-21T02:34:43.935Z)

```

1     """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3     One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
4     differ
5     only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
6     so
7     ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
8
9     Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
10    style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
11    ``AWS_SECRET_ACCESS_KEY``

```

```

9 / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10 (tests pass a fake; production lets the constructor build one via boto3).
11 """
12 from __future__ import annotations
13
14 import os
15 from typing import Any, Iterator, Optional
16
17 from backend.storage.base import StorageBackend
18 from backend.storage.ref import StorageRef, StorageStat
19
20
21 class S3Storage(StorageBackend):
22     def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
23                  region: Optional[str] = None, addressing_style: str = "auto",
24                  signed_url_ttl: int = 3600, **_ignored) -> None:
25         self._ttl = signed_url_ttl
26         if client is not None:
27             self._client = client
28         return
29         import boto3 # type: ignore # lazy: only when an s3:// URI is actually used
30         from botocore.config import Config # type: ignore
31         self._client = boto3.client(
32             "s3",
33             endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
34             region_name=region,
35             config=Config(s3={"addressing_style": addressing_style}),
36         )
37
38     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
39         if dst is None:
40             raise ValueError("S3Storage.fetch_to_local requires a dst path")
41         dst = str(dst)
42         if os.path.exists(dst) and not overwrite:
43             return dst
44         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
45         self._client.download_file(ref.bucket, ref.key, dst)
46         return dst
47
48     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
49         extra = {"ContentType": content_type} if content_type else None
50         self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
51         return ref
52
53     def exists(self, ref) -> bool:
54         try:
55             self._client.head_object(Bucket=ref.bucket, Key=ref.key)
56             return True
57         except Exception: # noqa: BLE001 - any boto/client error means "not found /
unreachable"
58             return False
59
60     def stat(self, ref) -> StorageStat:
61         h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
62         return StorageStat(size=int(h.get("ContentLength", 0)),
modified=h.get("LastModified"),
63                             etag=h.get("ETag"), content_type=h.get("ContentType"))
64
65     def list(self, prefix) -> Iterator[StorageRef]:
66         paginator = self._client.get_paginator("list_objects_v2")
67         for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
68             for obj in page.get("Contents", []) or []:
69                 key = obj["Key"]
70                 yield StorageRef("s3", prefix.bucket, key,
f"s3://{prefix.bucket}/{key}")

```

```

71
72     def delete(self, ref) -> None:
73         self._client.delete_object(Bucket=ref.bucket, Key=ref.key)
74
75     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
76         op = "get_object" if method.upper() == "GET" else "put_object"
77         return self._client.generate_presigned_url(
78             op, Params={"Bucket": ref.bucket, "Key": ref.key}, ExpiresIn=ttl or
self._ttl)
79

```

23. user (2026-06-21T02:34:44.140Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4     install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6     never from config. A ``client`` may be injected for tests.
7     """
8     from __future__ import annotations
9
10    from datetime import timedelta
11    from typing import Any, Iterator, Optional
12
13    from backend.storage.base import StorageBackend
14    from backend.storage.ref import StorageRef, StorageStat
15
16
17    class GCSStorage(StorageBackend):
18        def __init__(self, *, client: Any = None, project: Optional[str] = None,
19            signed_url_ttl: int = 3600, **_ignored) -> None:
20            self._ttl = signed_url_ttl
21            if client is not None:
22                self._client = client
23            return
24            from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25            self._client = storage.Client(project=project)
26
27        def _blob(self, ref: StorageRef):
28            return self._client.bucket(ref.bucket).blob(ref.key)
29
30        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31            if dst is None:
32                raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33            import os
34            dst = str(dst)
35            if os.path.exists(dst) and not overwrite:
36                return dst
37            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38            self._blob(ref).download_to_filename(dst)
39            return dst
40
41        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42            self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43            return ref
44
45        def exists(self, ref) -> bool:
46            return bool(self._blob(ref).exists())
47
48        def stat(self, ref) -> StorageStat:

```

```

49         b = self._blob(ref)
50         b.reload()
51         return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                             content_type=b.content_type)
53
54     def list(self, prefix) -> Iterator[StorageRef]:
55         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56             yield StorageRef("gcs", prefix.bucket, blob.name,
57 f"gcs://{prefix.bucket}/{blob.name}")
58
59     def delete(self, ref) -> None:
60         self._blob(ref).delete()
61
62     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
63         return self._blob(ref).generate_signed_url(
64             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)

```

24. user (2026-06-21T02:34:44.321Z)

```

1     """Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.
2
3     We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google
4     Drive
5     for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair
6     requirement ? it
7     needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on
8     access. So
9     ``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
10    **relative to
11    your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same
12    code the
13    ``local`` backend uses).
14
15    gdrive://Sharing/clip.mp4    ->    <mount>/My Drive/Sharing/clip.mp4
16
17    Why an alias instead of just a raw path? Portability: the mount root differs per machine
18    (Windows ``G:\\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My
19    Drive``), so the same
20    ``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**,
21    or set it
22    explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a
23    secret).
24
25    A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and
26    use
27    ``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend
28    intentionally does NOT
29    talk to the Drive API. When the mount (or a file in it) is missing, errors point the
30    user at the
31    fix: install Drive for Desktop and include the folder in syncing so it can be pulled
32    locally.
33
34    """
35    from __future__ import annotations
36
37    import glob
38    import os
39    import string
40    from typing import Iterator, Optional
41
42    from backend.storage.base import StorageBackend
43    from backend.storage.local import LocalStorage
44    from backend.storage.ref import StorageRef, StorageStat
45
46    _INSTALL_URL = "https://www.google.com/drive/download/"

```

```

34
35     # Shared remedy line ? how to make a folder available locally. Drive for Desktop streams
files,
36     # but a folder only appears once it's part of your synced Drive (and a *shared* folder
must first
37     # be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but
missing".
38     _SYNC_REMEDY = (
39         "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in
Google "
40         "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open
Drive in the "
41         "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'.
Drive then "
42         "downloads the bytes on first access."
43     )
44
45
46     def _not_mounted_msg(tried: Optional[str]) -> str:
47         return (
48             "Google Drive is not mounted, so the gdrive:// backend cannot resolve any
path.\n"
49             f"  Remedy:\n"
50             f"  1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
51             f"  2. Then {_SYNC_REMEDY}\n"
52             f"  3. If your 'My Drive' is not at the default location (Windows G:\\My Drive,
macOS "
53             f"~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "
54             f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
55         )
56
57
58     def _not_synced_msg(root: str, rel: str, uri: str) -> str:
59         return (
60             f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
61             f"(expected at {os.path.join(root, *rel.split('/')!r}).\n"
62             f"  This usually means the folder is not synced yet ? {_SYNC_REMEDY}\n"
63             f"  (Install/repair Drive for Desktop if needed: {_INSTALL_URL})"
64         )
65
66
67     def _autodetect_mount_root() -> Optional[str]:
68         """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google
Drive for
69         Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and
Windows
70         ``<letter>:\\My Drive`` (``G:`` is the install default). Linux/other ? configure
explicitly."""
71         home = os.path.expanduser("~")
72         for p in sorted(glob.glob(os.path.join(home, "Library", "CloudStorage",
"GoogleDrive*", "My Drive"))):
73             if os.path.isdir(p):
74                 return p
75         if os.name == "nt":
76             # G is the Drive-for-Desktop default; then scan the rest of the assignable
letters.
77             for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
78                 p = f"{letter}:\\My Drive"
79                 if os.path.isdir(p):
80                     return p
81         return None
82
83
84     class GDriveStorage(StorageBackend):
85         def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:

```

```

86         root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or
_autodetect_mount_root()
87         if not root or not os.path.isdir(root):
88             raise RuntimeError(_not_mounted_msg(root))
89         self._root = os.path.abspath(root)
90         self._local = LocalStorage(base_dir=self._root) # all I/O is plain local-FS
under the mount
91
92     def _local_ref(self, ref: StorageRef) -> StorageRef:
93         """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path
relative to
94         the mount root (``LocalStorage`` joins it under ``base_dir``)."""
95         rel = ref.key.lstrip("/")
96         key = os.path.join(*rel.split("/")) if rel else ""
97         return StorageRef("local", "", key, f"local://{key}")
98
99     def _require_present(self, ref: StorageRef) -> None:
100         """Guard the read paths: a path missing from the mount almost always means the
folder
101         isn't synced ? raise with that remedy instead of a cryptic FileNotFoundError
downstream."""
102         rel = ref.key.lstrip("/")
103         abs_path = os.path.join(self._root, *rel.split("/")) if rel else self._root
104         if not os.path.exists(abs_path):
105             raise FileNotFoundError(_not_synced_msg(self._root, rel, ref.uri))
106
107     def _to_gdrive_uri(self, abs_path: str) -> str:
108         """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""
109         rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
110         return f"gdrive://{rel}"
111
112     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
113         # The file is already local (via the mount): no dst => identity passthrough to
the mount
114         # path; a dst => copy out (a byte-stable scratch copy), exactly as the local
backend does.
115         self._require_present(ref)
116         return self._local.fetch_to_local(self._local_ref(ref), dst,
overwrite=overwrite)
117
118     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
119         self._local.put(local_path, self._local_ref(ref), content_type=content_type)
120         return ref
121
122     def exists(self, ref) -> bool:
123         return self._local.exists(self._local_ref(ref))
124
125     def stat(self, ref) -> StorageStat:
126         self._require_present(ref)
127         return self._local.stat(self._local_ref(ref))
128
129     def list(self, prefix) -> Iterator[StorageRef]:
130         for r in self._local.list(self._local_ref(prefix)):
131             uri = self._to_gdrive_uri(r.key)
132             yield StorageRef("gdrive", "", uri[len("gdrive://"):], uri)
133
134     def delete(self, ref) -> None:
135         self._local.delete(self._local_ref(ref))
136

```

25. assistant (2026-06-21T02:34:46.456Z)

Now let me search for how detector_impl and AI handoff selection happens:

26. user (2026-06-21T02:34:55.240Z)

```
artifacts/so.py
backend/config/models.py
backend/pipeline/detectors/native_wasb_runner.py
backend/pipeline/detectors/stub_runner.py
backend/pipeline/segmenters/fusion_hybrid.py
backend/pipeline/segmenters/trajectory_hybrid.py
backend/pipeline/segmenters/wasb_hybrid.py
backend/worker/__main__.py
tests/test_native_wasb_runner.py
tests/test_stub_runner.py
```

27. user (2026-06-21T02:34:57.328Z)

```
1      """Shared engine for trajectory-substrate hybrid segmenters.
2
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5      now the single home for the whole pipeline shape:
6
7          healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8          -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10     A concrete subclass supplies only its identity and runner wiring:
11     - ``family``          ? config section under ``indexing.*``, output subdir, and
12                           the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                                   detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20     import os
21     import time
22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47         #: config section under `indexing` (velocity_threshold lives there), the
48         #: output subdir for trajectories, and the metadata detector-tag prefix.
49         family: str = ""
50         #: human-readable label used in log lines.
51         display_label: str = ""
```

```

52
53     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54         """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
55         raise NotImplementedError
56
57     def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
58         """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
59         native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
60         raise NotImplementedError(
61             f"detector_impl='native' is not supported for the "
62             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
63             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
64
65     def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
66         """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
67
68         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
69         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73         (the WSL runner) so the default production path is unchanged
74         (``docs/DECOUPLED_COMPUTE/``).
75         """
76         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77         if impl == "stub":
78             from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
self.display_label or "trajectory")
80             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
81         if impl in ("native", "native_wasb", "wasb_native"):
82             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
self.display_label or "trajectory")
83             return self._build_native_runner(idx_cfg)
84         return self._build_runner(idx_cfg)
85
86     @staticmethod
87     def _probe_fps_width(video_path: str) -> tuple:
88         """Return (fps, frame_width) for a video, with sensible fallbacks."""
89         cap = cv2.VideoCapture(video_path)
90         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
91         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
92         cap.release()
93         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
94
95     @staticmethod
96     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
97         """Provider-reported exchange count, else a duration-based estimate.
100
101         One canonical implementation ? the twins had drifted (wasb relied on

```



```

102         ``int(None)`` raising into the fallback; same result, by accident).
103         """
104         shot_count = segment.get("shuttle_exchange_count")
105         if shot_count is None:
106             return max(3, int(duration / 0.8))
107         try:
108             return int(shot_count)
109         except (ValueError, TypeError):
110             return max(3, int(duration / 0.8))
111
112     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
113     -----
114     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
115                                frame_width: float, video_path: str,
116                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
117         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
118         BASE
119         returns exactly the 4 motion keys, so the trajectory twins persist byte-
120         identical
121         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
122         features). ``points`` are the whole-video trajectory points
123         (TrajectoryPoint)."""
124         return {
125             "peak_velocity": cw["peak_velocity"],
126             "mean_velocity": cw["mean_velocity"],
127             "active_frame_count": cw["active_frame_count"],
128             "track_id_count": cw["track_id_count"],
129         }
130
131     def _select_for_handoff(self, windows: List[Dict[str, Any]],
132                           window_stats: List[Dict[str, Any]],
133                           idx_cfg: Dict[str, Any]) -> List[int]:
134         """Indices of the candidate windows that proceed to the AI handoff (and thus may
135         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
136         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
137         Persisted candidates are NOT affected ? they remain the full superset for
138         offline
139         re-scoring."""
140         return list(range(len(windows)))
141
142     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
143                      player_pool: Optional[List[str]] = None,
144                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
145         # override-ignore: the ABC declares the Result contract (Success|Failure)
146         # but every live segmenter still returns a bare list ? the documented
147         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
148         label = self.display_label
149         # --- Sport profile + AI provider wiring (identical across segmenters) ---
150         sport_name = self.config.get("sport", "badminton")
151         try:
152             sport_profile = sport_registry.get(sport_name)
153         except KeyError as e:
154             logger.error(str(e))
155         return []

```

28. user (2026-06-21T02:34:58.489Z)

```

.claude/worktrees/boxhill17/backend/config/models.py
.claude/worktrees/boxhill17/backend/pipeline/segmenters/trajectory_hybrid.py
.claude/worktrees/boxhill17/tests/test_config.py
.claude/worktrees/boxhill17/tests/test_generate_highlights.py
.claude/worktrees/boxhill17/tests/test_tracknet_hybrid.py
.claude/worktrees/boxhill17/tools/generate_highlights.py
.claude/worktrees/gap-doc/backend/config/models.py
.claude/worktrees/gap-doc/backend/pipeline/segmenters/trajectory_hybrid.py

```

```

.claude/worktrees/gap-doc/tests/test_config.py
.claude/worktrees/gap-doc/tests/test_generate_highlights.py
.claude/worktrees/gap-doc/tests/test_tracknet_hybrid.py
.claude/worktrees/gap-doc/tools/generate_highlights.py
.claude/worktrees/hardchop/backend/config/models.py
.claude/worktrees/hardchop/backend/pipeline/segmenters/trajectory_hybrid.py
.claude/worktrees/hardchop/tests/test_config.py
.claude/worktrees/hardchop/tests/test_generate_highlights.py
.claude/worktrees/hardchop/tests/test_tracknet_hybrid.py
.claude/worktrees/hardchop/tools/generate_highlights.py
.claude/worktrees/pr-stitch/backend/config/models.py
.claude/worktrees/pr-stitch/backend/pipeline/segmenters/trajectory_hybrid.py
.claude/worktrees/pr-stitch/tests/test_config.py
.claude/worktrees/pr-stitch/tests/test_generate_highlights.py
.claude/worktrees/pr-stitch/tests/test_tracknet_hybrid.py
.claude/worktrees/pr-stitch/tools/generate_highlights.py
.claude/worktrees/recdoc/backend/config/models.py
.claude/worktrees/recdoc/backend/pipeline/segmenters/trajectory_hybrid.py
.claude/worktrees/recdoc/tests/test_config.py
.claude/worktrees/recdoc/tests/test_generate_highlights.py
.claude/worktrees/recdoc/tests/test_tracknet_hybrid.py
.claude/worktrees/recdoc/tools/generate_highlights.py
.claude/worktrees/resweep/backend/config/models.py
.claude/worktrees/resweep/backend/pipeline/segmenters/trajectory_hybrid.py
.claude/worktrees/resweep/tests/test_config.py
.claude/worktrees/resweep/tests/test_generate_highlights.py
.claude/worktrees/resweep/tests/test_tracknet_hybrid.py
.claude/worktrees/resweep/tools/generate_highlights.py
.claude/worktrees/tier1-windowing/backend/config/models.py
.claude/worktrees/tier1-windowing/backend/pipeline/segmenters/trajectory_hybrid.py
.claude/worktrees/tier1-windowing/tests/test_config.py
.claude/worktrees/tier1-windowing/tests/test_generate_highlights.py
.claude/worktrees/tier1-windowing/tests/test_tracknet_hybrid.py
.claude/worktrees/tier1-windowing/tools/generate_highlights.py
.claude/worktrees/w2-safeguard/backend/config/models.py
.claude/worktrees/w2-safeguard/backend/pipeline/segmenters/trajectory_hybrid.py
.claude/worktrees/w2-safeguard/tests/test_config.py
.claude/worktrees/w2-safeguard/tests/test_generate_highlights.py
.claude/worktrees/w2-safeguard/tests/test_tracknet_hybrid.py
.claude/worktrees/w2-safeguard/tools/generate_highlights.py
artifacts/so.py
backend/config/models.py
backend/pipeline/segmenters/trajectory_hybrid.py
backend/worker/__main__.py
scratch/extract_19june.py
tests/test_config.py
tests/test_generate_highlights.py
tests/test_tracknet_hybrid.py
tests/test_worker.py
tools/generate_highlights.py

```

29. assistant (2026-06-21T02:35:00.842Z)

Let me read more of the trajectory_hybrid to see the skip_ai_handoff logic and how main.py selects backends:

30. user (2026-06-21T02:35:01.491Z)

```

150         return []
151
152         idx_cfg = self.config.get("indexing", {})
153         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
154         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is

```

```

155         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
156         # measure it against the Gemini path, or to avoid the cloud entirely). With
157         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
158         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
159         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
160         if skip_ai:
161             model_name = "local-noai"
162             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
163                         "be emitted as rallies; no %s handoff, no API key needed.",
164                         label, provider_name)
165         else:
166             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
167             api_key_env_var = f"{provider_name.upper()}_API_KEY"
168             api_key = os.environ.get(api_key_env_var)
169             if not api_key:
170                 from backend.pipeline.segmenters._keyhint import api_key_hint
171                 logger.error(
172                     f"{api_key_env_var} environment variable is not set! "
173                     f"To run the {provider_name} handoff, set your API key. "
174                     + api_key_hint(api_key_env_var)
175                 )
176                 return []
177             try:
178                 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
179             except KeyError as e:
180                 logger.error(str(e))
181                 return []
182
183         video_duration = get_video_duration(video_path)
184         if video_duration <= 0:
185             logger.error("Invalid video duration.")
186             return []
187         fps, frame_width = self._probe_fps_width(video_path)
188
189         # --- Runner config + windowing thresholds ---
190         # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
191         # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
192         # it delegates to the subclass _build_runner (the real WSL/native runner).
193         try:
194             runner, runner_params = self._resolve_runner(idx_cfg)
195         except NotImplementedError as e:
196             # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
197             # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
198             logger.error("%s: %s", label, e)
199             return []
200
201         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
202         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
203         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
204         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
205         max_window_duration = idx_cfg.get("max_window_duration", 0.0)
206         window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
207         window_hard_cap = idx_cfg.get("window_hard_cap", False)
208         window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
209         inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
210         inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)

```

```

211 window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
212 window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
213 handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
214 ai_max_workers = idx_cfg.get("ai_max_workers", 5)
215 log_candidates = idx_cfg.get("log_yolo_candidates", True)
216 store_candidates = idx_cfg.get("store_yolo_candidates", True)
217
218 detection_params = {
219     "detector": self.name,
220     "velocity_thresh": velocity_thresh,
221     "merge_gap": merge_gap,
222     "min_action_window_duration": min_window_duration,
223     **runner_params,
224 }
225
226 # --- Phase 1: env healthcheck (fail fast with a clear message) ---
227 ok, msg = runner.healthcheck()
228 if not ok:
229     logger.error("%s detector environment not ready: %s", label, msg)
230     return []
231 logger.info("%s detector env OK: %s", label, msg)
232
233 # --- Phase 2: trajectory -> candidate rally windows ---
234 # Trajectory detectors process the whole video in one pass (they carry
235 # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
236 t_start = time.time()
237 out_dir = os.path.join("output", self.family)
238 csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
239 if not csv_path:
240     logger.error("%s produced no trajectory; aborting.", label)
241     return []
242
243 points = runner.parse_trajectory_csv(csv_path)
244 logger.info("Parsed %d trajectory points (%d visible).",
245             len(points), sum(1 for p in points if p.visible))
246
247 all_candidate_windows = runner.trajectory_to_action_windows(
248     points, fps=fps, frame_width=frame_width, chunk_start=0.0,
249     velocity_thresh=velocity_thresh, merge_gap=merge_gap,
250     min_window_duration=min_window_duration, chunk_index=0,
251     max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
252     window_hard_cap=window_hard_cap,
253     window_inactivity_end=window_inactivity_end,
254     inactivity_rally_thresh=inactivity_rally_thresh,
255     inactivity_min_density=inactivity_min_density,
256     window_blob_fallback=window_blob_fallback,
257     window_blob_factor=window_blob_factor,
258 )
259 logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
260             label, time.time() - t_start, len(all_candidate_windows))
261
262 if log_candidates:
263     for cw in all_candidate_windows:
264         logger.info(f"[{label}] rally
265 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
266                  f"({cw['end'] - cw['start']:.1f}s) |
267 frames={cw['active_frame_count']} "
268                  f"peak_v={cw['peak_velocity']:.3f}
269 mean_v={cw['mean_velocity']:.3f}")
270
271 # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
272 fusion
273 # segmenter adds net_crossings + person-cue features. Computed once, reused for
274 both
275 # persistence and the handoff gate below.

```

```

271         window_stats = [
272             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
273             for cw in all_candidate_windows
274         ]
275
276         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
277         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
278         if store_candidates:
279             for i, cw in enumerate(all_candidate_windows):
280                 candidate_ids[i] = self.db.add_candidate_segment(
281                     video_id, detector=self.name,
282                     start_time=cw["start"], end_time=cw["end"],
283                     chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
284                     stats=window_stats[i], detection_params=detection_params,
285                 )
286
287         if not all_candidate_windows:
288             return []
289
290         # --- Phase 3: AI provider handoff over padded candidate clips ---
291         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
292         # candidates above stay the full superset ? only the served play_segments are
gated.
293         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
294
295         ai_segments: List[dict] = []
296         if skip_ai:
297             # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
298             # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
299             # falls back to the duration-based estimate (no AI exchange count).
300             ai_segments = [
301                 {
302                     "start_time": all_candidate_windows[wi]["start"],
303                     "end_time": all_candidate_windows[wi]["end"],
304                     "shuttle_exchange_count": None,
305                     "shot_count_confidence": "LocalNoAI",
306                     "_candidate_id": candidate_ids[wi],
307                 }
308                 for wi in handoff_indices
309             ]
310             logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
311                         "(no AI handoff).", len(ai_segments))
312         else:
313             handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
314             os.makedirs(handoff_dir, exist_ok=True)
315             logger.info("Phase 3: %s validation on %d candidate windows...",
316                         provider_name, len(handoff_indices))
317
318             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
319                 w_start = max(0, window["start"] - handoff_padding)
320                 w_end = min(video_duration, window["end"] + handoff_padding)
321                 w_dur = w_end - w_start
322                 w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
323                 if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
324                     return []
325                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
326                 segments, _ = provider.analyze_video(w_path, prompt,

```

```

chunk_duration=w_dur,
327                                     label=f"Handoff {wi+1}")
328         valid = []
329         for seg in segments:
330             try:
331                 seg["start_time"] = float(seg["start_time"]) + w_start
332                 seg["end_time"] = float(seg["end_time"]) + w_start
333                 seg["_candidate_id"] = candidate_ids[wi]
334                 valid.append(seg)
335             except (ValueError, TypeError, KeyError) as e:
336                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
337             try:
338                 os.remove(w_path)
339             except OSError:
340                 pass
341             return valid
342
343         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
344             futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
345                         for wi in handoff_indices]
346             for future in futures:
347                 ai_segments.extend(future.result())
348             try:
349                 os.rmdir(handoff_dir)
350             except OSError:
351                 pass
352             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
353
354         # --- Phase 4: DB population + candidate linking ---
355         segments_data = []
356         for segment in ai_segments:
357             try:
358                 start = float(segment["start_time"])
359                 end = float(segment["end_time"])
360                 dur = end - start
361                 metadata = {
362                     "shot_count": self._shot_count_from(segment, dur),
363                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
364                     "detector": f"{self.family}-hybrid-{model_name}",
365                 }
366                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
367                 if segment_id:
368                     candidate_id = segment.get("_candidate_id")
369                     if candidate_id is not None:
370                         self.db.link_candidate_to_segment(candidate_id, segment_id)
371                     segments_data.append({
372                         "id": segment_id, "start_time": start, "end_time": end,
373                         "duration": dur, "metadata": metadata,
374                     })
375             except Exception as e:
376                 logger.warning("Skipping invalid segment struct: %s", e)
377
378         return segments_data
379

```

31. user (2026-06-21T02:35:01.971Z)

```

skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))

```

```

    if skip_ai:
        model_name = "local-noai"
        logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
                    "be emitted as rallies; no %s handoff, no API key needed.",
                    label, provider_name)
    else:
        model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
        api_key_env_var = f"{provider_name.upper()}_API_KEY"
        api_key = os.environ.get(api_key_env_var)
        if not api_key:
            from backend.pipeline.segmenters._keyhint import api_key_hint
            logger.error(
                f"{api_key_env_var} environment variable is not set! ")

```

32. assistant (2026-06-21T02:35:03.845Z)

Now let me check the main CLI entry point:

33. user (2026-06-21T02:35:04.843Z)

```

./claude/worktrees/agent-a771ffd85e18f2ed/backend/main.py
./claude/worktrees/bhi-execpolicy/backend/main.py
./claude/worktrees/boxhill17/backend/main.py
./claude/worktrees/gap-doc/backend/main.py
./claude/worktrees/hardchop/backend/main.py
./claude/worktrees/pr-stitch/backend/main.py
./claude/worktrees/recdoc/backend/main.py
./claude/worktrees/resweep/backend/main.py
./claude/worktrees/tier1-windowing/backend/main.py
./claude/worktrees/w2-safeguard/backend/main.py
./backend/main.py

```

34. user (2026-06-21T02:35:08.626Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13    logging.basicConfig(
14        level=logging.INFO,
15        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16        datefmt='%H:%M:%S'
17    )
18    logger = logging.getLogger(__name__)
19    from fastapi import FastAPI, HTTPException
20    from fastapi.middleware.cors import CORSMiddleware
21    from pydantic import BaseModel
22    from typing import List, Dict, Any, Optional
23
24    from fastapi.staticfiles import StaticFiles
25    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26    from backend.pipeline.validators.base import registry
27    from backend.infrastructure.database import Database
28    from backend.pipeline.segmenters.base import segmenter_registry

```

```

29     from backend.pipeline.stitcher.compiler import VideoCompiler
30     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32     from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
33     from backend.results import Failure # standardized error/result contract
34     from backend.reporting import emit_indexer_report
35
36     app = FastAPI(title="Sports Highlight Indexer API")
37
38     # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
39     # rejected by browsers per the fetch spec and is an unsafe default to carry into the
40     # auth/tenancy work; this tool uses no cookies, so credentials stay off.
41     app.add_middleware(
42         CORSMiddleware,
43         allow_origins=["*"],
44         allow_credentials=False,
45         allow_methods=["*"],
46         allow_headers=["*"],
47     )
48
49     # Serves static frontend files directly (now under api/ per approved structure)
50     os.makedirs("backend/api/static", exist_ok=True)
51     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
52
53     @app.get("/", response_class=HTMLResponse)
54     def serve_index():
55         index_path = "backend/api/static/index.html"
56         if os.path.exists(index_path):
57             with open(index_path, "r", encoding="utf-8") as f:
58                 return f.read()
59         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
60
61     # Global variables initialized at startup
62     db_instance: Optional[Database] = None
63     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
64
65     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
66     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
67     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
68     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
69     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
70     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
71     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
72     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
73         JobWaiting,
74         _arm_wake_timer,
75         _drain_due_jobs,
76         _get_executor,
77         _job_to_request,
78         _MAX_DRAIN_WAKE_SEC,
79         _run_claimed_job,
80         _schedule_next_drain_if_waiting,
81     )
82
83     # Legacy thin wrapper for any remaining direct calls during transition.
84     # New code should import load_app_config / AppConfig from backend.config directly.
85     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
86         cfg = load_app_config(config_path)

```



```

87         return cfg.model_dump(mode="json")
88
89     # --- API Models ---
90     class ProcessRequest(BaseModel):
91         video_path: str
92         player_pool: Optional[List[str]] = None
93         max_frames: Optional[int] = None
94         segmenter: Optional[str] = None
95
96     class QueryRequest(BaseModel):
97         video_id: str
98         server: Optional[str] = None
99         receiver: Optional[str] = None
100         duration_min: Optional[float] = None
101         event_type: Optional[str] = None
102
103     class CompileRequest(BaseModel):
104         video_id: str
105         segment_ids: List[int]
106         output_filename: str
107
108     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
109         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
110
111         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
112         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
113         and
114         keep the prior good results intact ? a failed/empty re-run never destroys prior
115         data.
116         """
117         if segments:
118             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
119         else:
120             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
121
122     # --- API Endpoints ---
123     @app.get("/api/config")
124     def get_config():
125         return global_config
126
127     @app.get("/api/health")
128     def health():
129         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
130         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
131         confirm the engine is reachable and which detector is wired. Never raises."""
132         sport = getattr(global_config, "sport", None)
133         indexing = getattr(global_config, "indexing", None)
134         return {
135             "status": "ok",
136             "sport": sport,
137             "default_segmenter": getattr(indexing, "default_segmenter", None),
138         }
139
140     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
141         """The full hash ? validate ? segment ? report pipeline for one video.
142
143         This is the former synchronous /api/process body, unchanged in behavior, now run on
144         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
145         the sync endpoint used to return (UI compatibility); the per-video observability
146         report is still emitted in `finally:` and grafted as response["reports"].
147
148         `progress` is an optional callable(stage: str) used to surface coarse job progress.
149         Raises on unexpected internal errors ? the caller records those as a job failure
150         (after this function has already restored the pre-run segments, see below).

```

```

150     """
151     notify = progress or (lambda stage: None)
152
153     notify("hashing")
154     video_id = compute_video_id(req.video_path)
155     was_reingest = db_instance.get_video(video_id) is not None
156
157     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
158     notify("validating")
159     logger.info(f"Running validations for {req.video_path}...")
160     val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
161     failures = [r for r in val_results if not r.passed]
162
163     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
164     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
165     seg_name = req.segmenter or default_seg
166     segmenter_actual = None
167     segmentation_ran = False
168     response: Optional[Dict[str, Any]] = None
169     try:
170         if failures:
171             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
172             # status; it no longer destroys the video's prior good segments.
173             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
174             response = {
175                 "video_id": video_id,
176                 "status": "failed",
177                 "message": "Video failed validation checks.",
178                 "results": [r.model_dump() for r in val_results],
179             }
180             return response
181
182     # 2. Run segmenter & indexer
183     logger.info("[API] Video passed checks. Segmenting and indexing...")
184     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
185
186     segmenter_cls = segmenter_registry.get(seg_name)
187     if not segmenter_cls:
188         # Submit-time validation already 400s unknown names; this is the defensive
189         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
190         available = list(segmenter_registry.list_available().keys())
191         response = {
192             "video_id": video_id,
193             "status": "failed",
194             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
195             "results": [r.model_dump() for r in val_results],
196             "play_segments": [],
197         }
198         return response
199
200     logger.info(f"[API] Using segmenter: {seg_name}")

```

35. assistant (2026-06-21T02:35:10.872Z)

Now let me search for configuration loading and environment detection:

36. user (2026-06-21T02:35:11.632Z)

```

backend/config/models.py:233:     output_dir: str = "output"
backend/config/models.py:296:     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default
``local`` = today's
backend/config/models.py:303:     # Output root for produced artifacts. "" => fall back to
output.output_dir (non-breaking).
backend/config/models.py:311:     # Durable workspace root URI. "" => precedence: output_root,
then output.output_dir.
backend/config/models.py:312:     workspace_root: str = ""

```

37. user (2026-06-21T02:35:13.363Z)

```

1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     from pathlib import Path
14     from typing import Any
15
16     from pydantic import BaseModel
17
18     from .models import AppConfig
19
20     logger = logging.getLogger(__name__)
21
22     DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
""") -> list[str]:
26         """Dotted paths in `data` that the typed config will not honor.
27
28         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
29         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
30         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
31         user's intent is lost ? collect every such key so load_config can warn.
32         """
33         unknown: list[str] = []
34         fields = model_cls.model_fields
35         for key, val in data.items():
36             if key not in fields:
37                 unknown.append(prefix + key)
38                 continue
39             ann = fields[key].annotation
40             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
41                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
42         return unknown
43
44
45     def _get_builtin_defaults() -> dict[str, Any]:
46         """Return a plain dict of the current built-in defaults.
47
48         The AppConfig model is the single source of truth for defaults.
49         """
50         return AppConfig().model_dump(mode="json")
51
52

```

```

53     def load_config(path: str | Path | None = None) -> AppConfig:
54         """Load configuration with the following precedence:
55
56         1. Built-in defaults defined in the Pydantic models.
57         2. Values from the JSON file (if present and readable).
58         3. (Future) environment / CLI overrides.
59
60         Missing file or unreadable file ? returns a fully defaulted AppConfig.
61         Unknown keys in the file are tolerated (extra="allow" on the model).
62         """
63         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65         file_data: dict[str, Any] = {}
66         if cfg_path.exists():
67             try:
68                 with open(cfg_path, "r", encoding="utf-8") as f:
69                     file_data = json.load(f)
70             except Exception as exc:
71                 # Non-fatal: continue with defaults + whatever we could parse.
72                 # In production we might log here.
73                 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75                 # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
76                 # otherwise crash startup: a truthy non-mapping hits `.items()` in
`_unknown_key_paths`, and any
77                 # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
78                 # loader's "bad input is non-fatal" contract.
79                 if not isinstance(file_data, dict):
80                     logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
81                                 cfg_path, type(file_data).__name__)
82                 file_data = {}
83
84         if file_data:
85             unknown = _unknown_key_paths(file_data, AppConfig)
86             if unknown:
87                 logger.warning(
88                     "[CONFIG WARNING] %s contains keys the typed config does not "
89                     "recognize (typo'd or removed knobs ? top-level extras are kept "
90                     "but unread; unknown section keys are silently dropped): %s",
91                     cfg_path, ", ".join(sorted(unknown)),
92                 )
93
94         # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
95         merged = {**_get_builtin_defaults(), **file_data}
96
97         return AppConfig.model_validate(merged)
98
99
100     def save_default_config(path: str | Path | None = None) -> Path:
101         """Write a fresh default config.json.
102
103         Used by scripts/doctor.py diagnostics and the --setup CLI path.
104         Overwrites if the file exists.
105         """
106         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
107         cfg = AppConfig()
108         cfg_path.parent.mkdir(parents=True, exist_ok=True)
109         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
110         return cfg_path
111
112

```

```

113     # Backwards-compatible alias used by some older call sites during transition.
114     def get_default_config() -> dict[str, Any]:
115         """Return defaults as a plain dict (for transitional use only)."""
116         return _get_built_in_defaults()
117

```

38. assistant (2026-06-21T02:35:15.912Z)

Now let me search for how environment detection happens (Cloud Run vs local):

39. user (2026-06-21T02:35:22.596Z)

```

.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/loader.py:    3. (Future) environment /
CLI overrides.
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py:    conda_sh: str =
"~/miniconda3/etc/profile.d/conda.sh"
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py:    conda_sh: str =
"~/miniconda3/etc/profile.d/conda.sh"
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/batch.py:    # This keeps collector
manifest paths working in tests and mixed environments.
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gemini_refine.py: def
refine_window(api_key: str, model: str, sport_profile, video: Interval,
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gemini_refine.py:         prompt =
sport_profile.get_prompt(chunk_duration=ce - cs)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gemini_refine.py:         sport_profile =
sport_registry.get("badminton")
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gemini_refine.py:         futs =
[ex.submit(refine_window, api_key, model, sport_profile, video, w, pad, duration,
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gemini_refine.py:         sport_profile =
sport_registry.get("badminton")
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gemini_refine.py:         prompt =
sport_profile.get_prompt(chunk_duration=cend - cstart)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/tracknet_runner.py:
conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/tracknet_runner.py:
conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_runner.py:
conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_runner.py:
conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py:         # Get
sport profile config
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py:
sport_profile = sport_registry.get(sport_name)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py:
f"{api_key_env_var} environment variable is not set! "
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py:
# Get prompt from sport profile
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py:
prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py:         #
Get prompt from sport profile

```

40. assistant (2026-06-21T02:35:24.436Z)

Let me look for docs on deployment profiles:

41. user (2026-06-21T02:35:25.682Z)

```

./ .claude/worktrees/agent-a771ff1d85e18f2ed/docs/ANALYZERS_AND_RECONCILERS.md
./ .claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/checkpoints/2026-06-eval-
baseline/README.md
./ .claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/CODE_AUDIT_AND_TEST_HARDENING-
COMPLETED-2026-06-14.md

```

```
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/decisions/DECISIONS.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/DEFERRED_HARDENING_PLAN-
COMPLETED-2026-06-14.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/HARDENING_LOOP_HANDOFF-
COMPLETED-2026-06-14.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/MONETIZATION_AUDIT.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/2026-06-first-ab-
experiment/README.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/2026-06-human-gt-
tuning/README.md
xargs: grep: terminated by signal 13
```

42. user (2026-06-21T02:35:28.354Z)

```
total 1016
drwxr-xr-x 1 avidu 197609      0 Jun 21 06:18 .
drwxr-xr-x 1 avidu 197609      0 Jun 21 06:18 ..
-rw-r--r-- 1 avidu 197609   3926 Jun 15 14:11 ABLATION_LAYER.md
-rw-r--r-- 1 avidu 197609  52912 Jun 15 14:11 ANALYZERS_AND_RECONCILERS.md
-rw-r--r-- 1 avidu 197609  14849 Jun 19 21:46 BACKLOG.md
-rw-r--r-- 1 avidu 197609   1045 Jun 15 14:11 CODE_AUDIT_AND_TEST_HARDENING.md
-rw-r--r-- 1 avidu 197609  86874 Jun 19 21:46 CODE_MAP.md
-rw-r--r-- 1 avidu 197609  17636 Jun 15 14:11 COMMERCIALIZATION.md
-rw-r--r-- 1 avidu 197609   1010 Jun 15 14:11 DEFERRED_HARDENING_PLAN.md
-rw-r--r-- 1 avidu 197609  22597 Jun 19 13:03 DESKTOP_APP_PLAN.md
-rw-r--r-- 1 avidu 197609   7036 Jun 15 14:11 EVALUATION.md
-rw-r--r-- 1 avidu 197609   9046 Jun 15 14:11 EXECUTION_POLICY.md
-rw-r--r-- 1 avidu 197609  15952 Jun 15 14:11 EXPERIMENT_HARNESS.md
-rw-r--r-- 1 avidu 197609   3126 Jun 12 22:33 FIELD_VISIT_ANSWER_SHEET.md
-rw-r--r-- 1 avidu 197609   8944 Jun 12 22:33 FIELD_VISIT_PLAYBOOK.md
-rw-r--r-- 1 avidu 197609   4365 Jun 17 19:44 GOLDEN_DATA_SHARING.md
-rw-r--r-- 1 avidu 197609  34489 Jun 17 19:44 GOLDEN_REGRESSION_FIXTURES.md
-rw-r--r-- 1 avidu 197609   6240 Jun  2 22:52 GOLDEN_SET_IMPLEMENTATION.md
-rw-r--r-- 1 avidu 197609   8513 Jun  4 10:10 GOLDEN_SET_PHASE1_VIDEOS.md
-rw-r--r-- 1 avidu 197609  16664 Jun 13 22:54 GOLDEN_SET_VENDORING.md
-rw-r--r-- 1 avidu 197609   4800 Jun 19 21:46 GOLDEN_VIDEOS.md
-rw-r--r-- 1 avidu 197609   1296 Jun 15 14:11 HARDENING_LOOP_HANDOFF.md
-rw-r--r-- 1 avidu 197609   8457 Jun 20 23:04 HOSTING_PLAN.md
-rw-r--r-- 1 avidu 197609   4324 Jun 19 21:46 HOW_RALLY_DETECTION_WORKS.md
-rw-r--r-- 1 avidu 197609  14496 Jun 12 22:33 I18N_PLAN.md
-rw-r--r-- 1 avidu 197609  10456 Jun 15 20:53 KHELSUTRA_PRODUCT_OVERVIEW.md
-rw-r--r-- 1 avidu 197609  20431 Jun 19 21:46 MULTI_SIGNAL_FUSION_PLAN.md
-rw-r--r-- 1 avidu 197609  15654 Jun 19 21:46 NEXT_STEPS.md
-rw-r--r-- 1 avidu 197609  15261 Jun 10 14:14 OWNED_MODEL_IMPLEMENTATION_PLAN.md
-rw-r--r-- 1 avidu 197609  11766 Jun 15 14:11 PERSONALIZATION_PLAN.md
-rw-r--r-- 1 avidu 197609   4408 Jun 14 11:42 PER_VIDEO_OUTPUT_LAYOUT.md
-rw-r--r-- 1 avidu 197609  11890 Jun 20 23:02 PER_VIDEO_WORKSPACE.md
-rw-r--r-- 1 avidu 197609  49212 Jun 15 14:11 PLATFORM_ARCHITECTURE.md
-rw-r--r-- 1 avidu 197609  12808 Jun 12 22:33 PMF_FIELD_SIGNALS.md
-rw-r--r-- 1 avidu 197609  16871 Jun  8 07:11 PMF_MARKET_RESEARCH.md
-rw-r--r-- 1 avidu 197609  24523 Jun 15 14:11
POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
-rw-r--r-- 1 avidu 197609   4347 Jun 12 22:33 PRODUCT_FIELD_ASSOCIATE_FLYER.md
-rw-r--r-- 1 avidu 197609   3739 Jun 16 09:42 PROJECT_DOC_TEMPLATE.md
-rw-r--r-- 1 avidu 197609  16490 Jun 19 21:47 PROMINENT_COURT_DETECTION.md
-rw-r--r-- 1 avidu 197609  61721 Jun 19 21:46 QUALITY_ITERATIONS.md
-rw-r--r-- 1 avidu 197609   6457 Jun 18 13:12 R1_MILESTONE_LAUNCH_REPORT.md
-rw-r--r-- 1 avidu 197609  15067 Jun 19 21:46 R2_EVAL_METRIC_DESIGN.md
-rw-r--r-- 1 avidu 197609  16384 Jun 19 21:46 RALLY_DETECTION_GAP_CLOSURE.md
-rw-r--r-- 1 avidu 197609  32871 Jun 19 21:46 RALLY_DETECTION_QUALITY_REPORT.md
-rw-r--r-- 1 avidu 197609  45298 Jun 19 21:46 RALLY_QUALITY_RESEARCH.md
-rw-r--r-- 1 avidu 197609  10778 Jun 21 06:18 README.md
-rw-r--r-- 1 avidu 197609  16248 Jun 17 19:44 REAL_FOOTAGE_VALIDATION_REPORT.md
-rw-r--r-- 1 avidu 197609   8224 Jun 15 14:11 ROADMAP.md
```

```

-rw-r--r-- 1 avidu 197609 30436 Jun 15 14:11 ROORA_LABELING_GUIDELINES.md
-rw-r--r-- 1 avidu 197609 16120 Jun 20 23:05 SETUP_NEW_MACHINE.md
-rw-r--r-- 1 avidu 197609 11935 Jun 13 22:54 STORAGE_SHARING_MODEL.md
-rw-r--r-- 1 avidu 197609 5789 Jun 15 14:11 TRACKNET_WSL_SETUP.md
-rw-r--r-- 1 avidu 197609 6152 Jun 11 20:48 UI_ALPHA_EXECUTION_PLAN.md
-rw-r--r-- 1 avidu 197609 8503 Jun 11 00:35 UI_PROPOSAL.md
-rw-r--r-- 1 avidu 197609 11217 Jun 13 21:02 VIDEO_LOCALITY_MODEL.md
-rw-r--r-- 1 avidu 197609 8495 Jun 18 13:12 VIDEO_RECORDING_GUIDELINES.md
drwxr-xr-x 1 avidu 197609 0 Jun 15 14:11 archives
drwxr-xr-x 1 avidu 197609 0 Jun 11 00:35 mockups

```

43. assistant (2026-06-21T02:35:30.289Z)

Let me check the HOSTING_PLAN and PLATFORM_ARCHITECTURE docs:

44. user (2026-06-21T02:35:30.935Z)

```

1      # Hosting Plan ? khelsutra.guru
2
3      **Status:** PROPOSAL ? awaiting owner ratification.
4      **Date:** 2026-06-10 · **Constraints:** budget $10?25/mo · GCP credits available ·
DigitalOcean
5      acceptable · alpha backend = owner's GPU box behind a tunnel · 10?25 alpha testers ·
registrar = GoDaddy.
6
7      > **? Update (2026-06-20, ADR-013): DigitalOcean is DROPPED for COMPUTE** ? GCP is the
sole compute
8      > stack (training + serving). The ***"All-DigitalOcean"*** compute option below is
retired; DO Spaces
9      > remains only a *possible* S3-compatible storage target. Edge/site hosting choices here
are unaffected.
10     **Companion:** `docs/UI_PROPOSAL.md`.
11
12     ## 1. Recommendation in one line
13
14     **Alpha: free edge + your box** (Cloudflare DNS/Pages/Access/Tunnel ? FastAPI on the GPU
box) at
15     **$0/mo**, saving GCP credits; **Beta: GCP control plane + your box as pull-worker**
(Cloud Run +
16     GCS + Firebase Auth, credits-covered), keeping the edge where it's free.
17
18     Rationale for the Cloudflare pieces despite the GCP preference: GCP has no free
equivalent of the
19     Tunnel + Access combo (IAP requires a load balancer, ~$18/mo standing cost), and the
alpha needs
20     exactly three things GCP charges for but Cloudflare gives free: a secure tunnel to a
home box with
21     a custom domain, email-allowlist auth (free ?50 users ? fits 10?25 testers), and static
hosting.
22     Credits are better spent at beta on compute/storage, which is where GCP is excellent.
23
24     ## 2. Alpha topology ($0/mo)
25
26     ` ` `
27     tester browser
28     ? https
29     ?
30     Cloudflare edge (free): DNS + TLS + Access (email-OTP allowlist)
31     ??? app.khelsutra.guru ? Cloudflare Pages (React SPA; auto-deploy from GitHub)
32     ??? api.khelsutra.guru ? Cloudflare Tunnel (outbound-only `cloudflared` service on
the box)
33
34     ?
35     FastAPI :8000 on the GPU box (Windows)
36     ? Gemini segmenter path (no GPU in request path)

```

```

37         ? async jobs (#16), SQLite WAL, output/ on disk
38     ...
39
40     - **No inbound ports opened** on the home network; `cloudflared` dials out. Home IP
never exposed.
41     - **Auth:** Cloudflare Access in front of BOTH hostnames; backend trusts the
42     `Cf-Access-Authenticated-User-Email` header (verify the Access JWT for rigor) for per-
user scoping.
43     - **Upload limit:** free-tier proxy caps request bodies at **100 MB** ? SPA uploads in
?90 MB
44     chunks (UI_PROPOSAL B2). Match videos (1?4 GB) flow fine as chunks.
45     - **Box offline = service offline.** Acceptable for alpha; SPA shows a friendly
"processing engine
46     offline" banner (a `/api/health` ping). Beta removes this with the queue.
47     - **Costs:** Pages, Tunnel, Access, DNS = $0. Only the Gemini per-run cost
(~$0.05/match) and
48     domain renewal.
49
50     ## 3. Beta topology (GCP credits; ~$15?20/mo post-credits)
51
52     ...
53     browser ??? Cloudflare edge (free, unchanged) ??? app.khelsutra.guru (Pages)
54     ??? api.khelsutra.guru ??? Cloud Run
(FastAPI control plane, scale-to-zero)
55     ? jobs
table (Cloud SQL Postgres)
56     ? signed
resumable URLs
57     ?
58     GCS buckets
(uploads / reels, lifecycle ? cold)
59     ? pull
jobs, outbound-only
60     GPU box = first
`byo_worker` (PLATFORM P3)
61     ...
62
63     - Uploads go **browser ? GCS directly** (resumable signed URLs) ? no proxy body limit,
no home
64     bandwidth for ingest; the box pulls only what it processes (WASB/owned-model jobs),
Cloud Run
65     handles Gemini-path jobs entirely in-cloud.
66     - Auth graduates to **Firebase Auth** (free; Google + email sign-in; India-friendly
phone OTP
67     later); Access allowlist retires.
68     - Post-credits estimate: Cloud Run ~$0?5 (free tier: 2 M req + 180 K vCPU-s/mo, scale-
to-zero) .
69     GCS 100 GB ? $2.60 + egress $0.12/GB (reel downloads ? watch this line) . Cloud SQL
smallest
70     ? $10 ? **~$15?20/mo**, inside budget. Egress lever if it grows: serve reels from
Cloudflare R2
71     (zero egress) while keeping everything else GCP.
72
73     ## 4. Alternatives considered
74
75     | Option | Monthly (alpha ? beta) | Pros | Cons |
76     |---|---|---|---|
77     | **Recommended hybrid** (CF edge + box ? +GCP) | **$0 ? ~$0 w/ credits** (~$15?20
after) | Cheapest at every phase; credits used where they matter; maps to PLATFORM P2/P3 | Two
vendors (CF + GCP) |
78     | **All-DigitalOcean** | ~$11 ? ~$17 ($6 droplet + Spaces $5: 250 GiB + **1 TiB egress
included**) | One simple vendor; Spaces' included egress is the best video-download deal; per-
second billing | No credits; no scale-to-zero; still needs a tunnel-equivalent to reach the box
(run `frp`/WireGuard on the droplet) |
79     | **All-GCP from day 1** | ~$18+ ? credits | Single vendor, credits immediately | LB+IAP

```



```

standing cost just to gate an alpha; no free tunnel to the home box; credits burn on plumbing |
80      | **All-Cloudflare** (Pages+Workers+R2) | ~$0 ? ~$5 | Zero egress on R2; cheapest
forever | FastAPI doesn't run on Workers (API stays on box/Cloud Run anyway); ignores GCP
credits |
81
82      ## 5. Domain setup ? GoDaddy ? Cloudflare DNS (one-time, ~15 min, you)
83
84      1. Create a free Cloudflare account ? **Add a domain** ? `khelsutra.guru` ? Free plan.
Cloudflare
85          imports existing records and shows **two nameservers** (e.g.
`xxx.ns.cloudflare.com`).
86      2. GoDaddy ? **My Products ? khelsutra.guru ? Manage DNS ? Nameservers ? Change ?
87          "I'll use my own nameservers"** ? paste the two Cloudflare nameservers ? Save.
Propagation:
88          minutes to 24 h. (Registration/renewal stays at GoDaddy; only DNS moves. Optional
later:
89          transfer the registration to Cloudflare Registrar for at-cost .guru renewals.)
90      3. Records (I manage after takeover): `app` ? Pages project; `api` ? `**Cloudflare API token** (you create: dash ? My Profile ? API Tokens) | Zone:DNS:Edit,
Account:Pages:Edit, Account:Access:Apps:Edit, Account:Tunnel:Edit ? scoped to this zone | DNS
records, Pages deploys (wrangler), Access policies, tunnel config |
98      | **GitHub repo** `khelsutra` (private) | Pages connects once (dashboard, you; build
root = `web/`) ? every push auto-deploys | I write the SPA in your Projects folder; commits
pushed = deployed |
99      | **On the box** (one-time, I hand you exact commands) | `cloudflared` installed as a
Windows service + `config.yml` | The tunnel itself |
100     | **GCP (beta only)** | A `khelsutra` project + service account (Cloud Run admin,
Storage admin, Cloud SQL admin) | Control-plane deploys |
101
102     Day-to-day flow after setup: I build ? commit ? push ? Pages deploys the SPA
automatically;
103     API/tunnel/DNS changes via the scoped token. Secrets (Gemini key, Access policy) never
live in the
104     repo; `.env` on the box + Cloudflare/GCP dashboards.
105
106     ## 7. Go-live security checklist (alpha)
107
108     Access allowlist ON for both hostnames ? CORS narrowed to `https://app.khelsutra.guru` ?
rate
109     limit + per-user upload quota ? `security.allowed_video_root` set ? upload size cap +
extension
110     allowlist ? **rotate the Gemini API key** (NEXT_STEPS housekeeping ? it was shared in
chat) ?
111     confirm no router port-forwards ? weekly backup: SQLite DB + `output/` ? GCS bucket
(rclone
112     scheduled task; credits).
113
114     ## 8. Decisions needed from owner
115
116     1. Ratify the **hybrid recommendation** (CF free edge now, GCP credits at beta)? Or
force
117         single-vendor (DO / GCP) despite the cost notes above?
118     2. OK to move **DNS to Cloudflare** (registration stays at GoDaddy)?
119     3. Beta reel storage: **GCS (credits)** vs **R2 (zero egress)** vs **DO Spaces (1 TiB
included)** ?
120         can defer until beta Drop 2.
121     4. Approve the access grants in §6 so I can run the setup end-to-end.
122

```

123 *Ratified choices get recorded in `docs/archives/decisions/DECISIONS.md`.*
124

45. user (2026-06-21T02:35:31.213Z)

```
1     # Per-video workspace ? one folder per video, local?cloud identical
2
3     > **Status:** `DRAFT` ? `in progress` (owner-requested 2026-06-20) . **Owner:** Avi .
**Created:** 2026-06-20 . **Last updated:** 2026-06-20
4     > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5     > **Tracking anchors:** $7 progress tracker is the source of truth; pointer in memory
`current-state`.
6     > **Relation to existing docs:** peer-of `DECOUPLED_COMPUTE` (reinforces the
storage/compute seam) and the appliance track; extends `file-placement-conventions`.
7     > **Honesty note:** claims tagged `[verified]` (checked vs code by the mapping
workflow), `[design]` (proposed), `[researched]`.
8
9     ---
```

10

```
11     ## 0. TL;DR
12     Today a single video's artifacts are **scattered** ? `output/<family>/*.csv`,
`output/chunks_<provider>_handoff/`, `~/clips/<key>_frames/`, `<scratch>/outputs/<video_id>/`,
reels in `output/`, logs to stdout, plus DB rows. The mapping workflow found **56 write sites
across 4 subsystems** `[verified]`. Goal: **one folder per video, keyed by the MD5 `video_id`,
holding everything about that video** (working video copy/pointer, frames, trajectory,
candidates, intervals/report, reel, run log) **except its metadata, which stays in the DB**. The
root is a **storage URI**, so a **local** deployment and a **cloud (bucket)** deployment use the
*same relative layout* ? only the root differs ? easy to find, easy to repro, easy to clean.
Single most important caveat: this is a **default-OFF, phased** convergence (legacy flat layout
untouched until a Launch-Report flip) because the worker's bucket push prefix and the WSL/native
fast-scratch paths are load-bearing.
```

13

```
14     ## 1. Problem & goal
15     - **Why now:** approaching launch + the DECOUPLED/Cloud-Run serving track. The owner
wants per-video repro parity across local and cloud ("everything about a video resides in one
place; metadata in the DB"). Ties [[feedback-launch-quality-bar]] (serious observability) and
the Cloud-Run subproject (ADR-012).
16     - **Good looks like:** given a `video_id`, a single `ls <root>/<video_id>/` shows *all*
of its artifacts, identically whether `<root>` is a local dir or `gs://?`. Deleting that one
folder fully cleans the video (DB row aside).
17     - **Read to get current:** the $4 reuse map;
`backend/utils/hashing.py::compute_video_id`; `backend/storage/` (URI abstraction);
`backend/worker/__main__.py`; `backend/pipeline/segmenters/trajectory_hybrid.py`.
18
19     ## 2. Decisions locked
20     Defaults chosen to keep momentum; the ?-marked ones in $8 are still owner-confirmable.
```

21

	#	Decision	Source / date	Implication
22	1	D1	**Key = full MD5 `video_id`** for the folder name (`<root>/<video_id>/`).	owner ask, 2026-06-20 Matches `videos.id` / the storage join key; collision-proof. Artifact
23	2	D2	**Root is a storage URI**;	local & cloud share ONE relative layout (only the root differs). design, [[monetization-plan]] Reuses `backend/storage/` dispatch
24	3	D3	**Two roots:** a *durable* workspace root (may be `gs://`) for artifacts to keep;	a *fast local compute-scratch* mirror (always local) for frame caches/chunks. Compute writes locally, then **pushes durable artifacts** to the workspace. design + back-compat WSL/native frame extraction never writes to a cloud root (perf + correctness).
25	4	D4	**DB stays metadata-only** + ONE additive nullable `videos.workspace_uri` (v6).	`NULL` = legacy flat layout. design No artifact bytes in the DB; the pointer makes a video's folder discoverable.
26	5	D5	**Default-OFF behind a flag** (`storage.single_folder_per_video`);	converge in phases; flip via a **Launch Report**. [[launch-report-convention]] Zero regression risk

```

during convergence; the deployed bucket layout only changes on the gated flip. |
29 | D6 | **NOT per-video (stay outside the folder):** the `weights/<version>/` bundle
(version-keyed), the SQLite DB, the WSL/native `wasb_infer.py` staging copy (infra), and the
training corpus (`labels/`,`videos/`) + eval fixtures (shared inputs). | [verified] mapping |
The convention covers *per-video* artifacts only; corpus/eval/model paths are intentionally
excluded. |
30
31 ## 3. Foundation ? the artifact map `[verified]`
32 The mapping workflow (`wf_085a5118`, 5 agents) catalogued every per-video write site.
Consolidated:
33
34 | Artifact | Today | Keyed by | Target subfolder |
35 |---|---|---|---|
36 | Working video copy (cloud fetch) / source | `scratch/in.mp4`, `~/clips/<key>.<ext>` |
video_id | `source/` (copy) or `source/source.uri` (local pointer) |
37 | Trajectory CSV (wasb/predict/stub) | `output/<family>/<key>_*.csv` | video_id |
`detector/` |
38 | WASB detector cache (`det_raw.jsonl`,`manifest.json`) |
`<csv_dir>/<key>_wasb_wasbcache/` | video_id | `detector/<key>_wasbcache/` |
39 | Frame PNG cache | `~/clips/<key>_frames/`,`scratch/<key>_frames/` | video_id |
`tmp/` (**local-only**) |
40 | Handoff clips (AI validation) | `output/chunks_<provider>_handoff/` ** (hardcoded)** |
video_id+idx | `tmp/` (**local-only**) |
41 | YOLO person chunks | `output/chunks_yolo/` ** (hardcoded)** | video_id+idx | `tmp/`
(**local-only**) |
42 | Gemini chunks / temp-trim | `output/chunks/<video_id>/`,`
output/temp_trimmed_<video_id>.mp4` | video_id | `tmp/` (**local-only**) |
43 | `intervals.csv`,`report.json` (worker) | `scratch/outputs/<video_id>/` ?
`<out_uri>/outputs/<video_id>/` | video_id | `outputs/` |
44 | Highlight reel | `output/<basename>.mp4` | video_id (via job) | `reels/` |
45 | Per-segment compile clips | `tempdir/clip_<idx>.mp4` | seg idx | `tmp/` (**local-
only**) |
46 | Obsreport JSON | `output/<video_id>_report.json` | video_id | `outputs/` |
47 | Run log | stdout (not persisted) | ? | `logs/run.log` |
48 | Candidate/play segments, jobs, events | SQLite tables | video_id | **DB only**
(metadata) |
49
50 Two genuinely **hardcoded** dirs to kill: `output/chunks_<provider>_handoff` and
`output/chunks_yolo` (this subsumes the long-carried "de-hardcode `output`" item,
`trajectory_hybrid.py:237,313`).
51
52 ## 4. Design / architecture
53 **Target layout (per video):**
54 ```
55 <workspace_root>/<video_id>/
56   source/          # working copy (cloud-fetched) OR source.uri pointer (local)
57   detector/        # trajectory CSV(s) + <key>_wasbcache/ (det_raw.jsonl, manifest.json)
58   outputs/         # intervals.csv, report.json, <video_id>_report.json (obsreport)
59   reels/           # compiled highlight mp4(s)
60   logs/            # run.log (per-video processing log)
61   tmp/             # LOCAL-ONLY transient: frame PNGs, handoff/yolo/gemini chunks, compile
clips ? never pushed; cache-hygiene drops this
62   ```
63 **Outside the per-video folder** (D6): `weights/<version>/`,`the SQLite DB,
`wasb_infer.py` staging, `labels/`+`videos/` corpus, eval fixtures.
64
65 **Reuse map (what's new vs reused):**
66 - **NEW** `backend/utils/workspace.py` ? pure path helper: `video_workspace(root,
video_id, *, sub=None, name=None)` builds `**REUSE** `backend/storage/` (`parse_uri`/`fetch`/`put`/`list_uris`) for all durable
read/write; `compute_video_id` for the key; the existing `keep_frames`/`min_free_gb` cache-
hygiene; `tools/cleanup_caches.py` (retarget to `tmp/`).

```

```

68     - **Adopt** at the call sites in §7 ? each currently builds its own
`os.path.join("output", ?)`, they switch to the helper, gated by `single_folder_per_video`.
69
70     **Config:** `storage.workspace_root` (URI; default empty),
`storage.single_folder_per_video` (bool, default **False**), `storage.workspace_prefix` (default
`"workspaces"` for the cloud namespace). Root precedence: `storage.workspace_root` ?
`storage.output_root` ? `output.output_dir`.
71
72     ## 6. Honest limits ? what this does NOT do
73     - A green suite proves the **helper + gated wiring** in unit tests; it does **NOT**
prove the cloud bucket layout end-to-end until a real GPU run exercises the flag (deferred ? the
in-flight M2 run uses the *legacy* layout).
74     - It does **not** move corpus/eval/weights into the convention (D6) ? those are shared,
not per-video.
75     - It does **not** change the DB's role; metadata stays in SQLite.
76
77     ## 7. Deliverables & progress tracker ? **source of truth**
78     Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, off
`origin/master`, no stacking.
79
80     | ID | Deliverable | Depends on | Gated? | Status | PR |
81     |----|-----|-----|-----|-----|----|
82     | P0 | `backend/utils/workspace.py` helper +
`storage.workspace_root`/`single_folder_per_video`/`workspace_prefix` config + unit tests (no
wiring; no behavior change) | ? | No | ? | this PR |
83     | P1 | v6 DB migration: nullable `videos.workspace_uri` + setter/getter | P0 | No | ? |
? |
84     | P2 | Converge the **worker** (`cmd_infer`/`cmd_train` scratch + push) onto the
workspace, flag-gated | P1 | No | ? | ? |
85     | P3 | Converge **segmenter + detector** output dirs (kill `output/chunks_` hardcodes;
`detector/` + `tmp/`), flag-gated | P1 | No | ? | ? |
86     | P4 | Converge **frame cache / WSL+native staging / reels / obsreport / run log** onto
`tmp/+logs/+reels/+outputs/`, flag-gated | P3 | No | ? | ? |
87     | P5 | **Launch Report** + flip default ON + retarget `tools/cleanup_caches.py` + docs |
P2,P3,P4 | ? owner | ? | ? |
88     | P6 | Validate the flag on a real GPU run (local + a `gcs://` workspace) ? repro parity
| P5 | ? owner GPU | ? | ? |
89
90     ## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)*
91     **Proposed defaults applied (override if you disagree):**
92     1. **Source video: copy or pointer** ? **default:** cloud-fetched copy lands in
`source/`; a local source is referenced by a `source/source.uri` pointer (no duplication of
large files). ? confirm.
93     2. **Cloud root namespace** ? **default:**
`gs://<bucket>/<workspace_prefix>/<video_id>/` with `workspace_prefix="workspaces"` (keeps per-
video folders tidy beside `videos/`, `labels/`, `weights/`; the legacy `outputs/<video_id>/` stays
until P5). ? confirm (affects the deployed bucket contract).
94     3. **Push transient chunks/frames to cloud** ? **default:** **No** ? `tmp/` is local-only
(huge + ephemeral). nice-to-know.
95     4. **Migration of existing artifacts** ? **default:** **forward-only** (legacy flat
layout left in place; optional one-shot migrator later). nice-to-know.
96
97     ## 9. Definition of done
98     - [ ] Helper + config land with tests (P0).
99     - [ ] `videos.workspace_uri` migration (P1).
100    - [ ] All §3 per-video write sites route through the helper under the flag (P2?P4); the
two hardcoded `output/chunks_` dirs are gone.

```

46. assistant (2026-06-21T02:35:33.178Z)

Perfect! Now I have a comprehensive understanding. Let me search for any existing deployment profile or startup config concepts:

47. user (2026-06-21T02:35:35.128Z)

```
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/loader.py: 3. (Future) environment /
CLI overrides.
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py: conda_sh: str =
"~/miniconda3/etc/profile.d/conda.sh"
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py: conda_sh: str =
"~/miniconda3/etc/profile.d/conda.sh"
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py: # The CLI's --output-dir
overrides this at startup (see backend/main.py::main).
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/batch.py: # This keeps collector
manifest paths working in tests and mixed environments.
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/infrastructure/database.py: hang.
Called once at startup. Returns the number of jobs swept.""
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/main.py:# Global variables initialized at
startup
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/tracknet_runner.py:
conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/tracknet_runner.py:
conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_runner.py:
conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_runner.py:
conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py: # Get
sport profile config
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py:
f"{api_key_env_var} environment variable is not set! "
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py:
# Get prompt from sport profile
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py: #
Get prompt from sport profile
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/trajectory_hybrid.py:
# --- Sport profile + AI provider wiring (identical across segmenters) ---
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/trajectory_hybrid.py:
f"{api_key_env_var} environment variable is not set! "
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/trajectory_hybrid.py:
logger.error("%s WSL environment not ready: %s", label, msg)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/yolo_hybrid.py: #
Get sport profile config
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/yolo_hybrid.py:
f"{api_key_env_var} environment variable is not set! "
```

48. assistant (2026-06-21T02:35:37.414Z)

Now let me check what the actual current state is in the main CLI:

49. user (2026-06-21T02:35:37.845Z)

```
200         logger.info(f"[API] Using segmenter: {seg_name}")
201         notify(f"segmenting ({seg_name})")
202         segmenter_actual = seg_name
203         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
204         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
205         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
206         segmenter = segmenter_cls(db_instance, global_config)
207         try:
208             result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
209         except Exception:
210             # A raising segmenter must not leave half-written rows: restore the pre-run
211             # state (same destroy-after-confirm contract as the Failure branch), then
let
212             # the job worker record the failure.
213             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
```

```

214         raise
215     segmentation_ran = True
216
217     if isinstance(result, Failure):
218         logger.error(f"[API] Segmenter failed: {result.message}")
219         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
220         response = {
221             "video_id": video_id,
222             "status": "failed",
223             "message": f"Segmenter '{seg_name}' failed: {result.message}",
224             "results": [r.model_dump() for r in val_results],
225             "play_segments": [],
226         }
227         return response
228
229     segments = result.value if hasattr(result, "value") else result
230     notify("finalizing")
231     _finalize_reingest(video_id, segments, play_wm, cand_wm)
232
233     response = {
234         "video_id": video_id,
235         "status": "success",
236         "message": f"Successfully indexed {len(segments)} play segments using
237 '{seg_name}' segmenter.",
238         "results": [r.model_dump() for r in val_results],
239         "play_segments": segments,
240     }
241     return response
242
243     finally:
244         # Always emit the per-video observability report (next to the video, or to
245         # report.output_dir). Never raises. Surface the paths in the response.
246         paths = emit_indexer_report(
247             db=db_instance, video_id=video_id, video_path=req.video_path,
248             config=global_config,
249             val_results=val_results, val_context=val_context,
250             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
251             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
252         )
253         if paths and isinstance(response, dict):
254             response["reports"] = paths
255
256
257 @app.post("/api/process", status_code=202)
258 def process_video(req: ProcessRequest):
259     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
260
261     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
262     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
263     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
264     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
265     """
266     allowed_root = getattr(getattr(global_config, "security", None),
267                             "allowed_video_root", None)
268     try:
269         validate_input_path(req.video_path, allowed_root=allowed_root)
270     except ValueError as e:
271         raise HTTPException(status_code=400, detail=str(e)) from e
272     if not os.path.exists(req.video_path):
273         raise HTTPException(status_code=404, detail=f"Video file not found at
274 '{req.video_path}'")
275     if req.segmenter and not segmenter_registry.get(req.segmenter):
276         available = list(segmenter_registry.list_available().keys())
277         raise HTTPException(
278             status_code=400,
279             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:

```

```

{available}"
275         )
276
277     job_id = uuid.uuid4().hex
278     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
279                                 req.player_pool, req.max_frames):
280         raise HTTPException(status_code=500, detail="Failed to persist job record.")
281     _get_executor().submit(_drain_due_jobs)
282     return JSONResponse(
283         status_code=202,
284         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
285     )
286
287
288 @app.get("/api/jobs/{job_id}")
289 def get_job(job_id: str):
290     """Job state: {status, progress, result, reports}. `status` = execution state
291     (queued|running|done|failed); the pipeline verdict is result["status"]."""
292     job = db_instance.get_job(job_id)
293     if not job:
294         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
295     result = job.get("result")
296     return {
297         "job_id": job["id"],
298         "status": job["status"],
299         "progress": job.get("progress"),
300         "video_id": job.get("video_id"),
301         "error": job.get("error"),
302         "result": result,
303         "reports": (result or {}).get("reports"),
304         "created_at": job.get("created_at"),
305         "started_at": job.get("started_at"),
306         "finished_at": job.get("finished_at"),
307     }
308
309 # --- Chunked upload (B2, PR-2) -----
310 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
311 in
312 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
313 via
314 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
315 # sequential-part logic, and abort cleanup ? is unchanged.
316 from backend.api import uploads as uploads_api
317
318 app.include_router(uploads_api.router)
319
320 # --- Highlight download (B3, PR-2) -----
321
322 @app.get("/api/highlights/{video_id}/{filename}")
323 def download_highlight(video_id: str, filename: str):
324     """Stream a compiled highlight reel ? `filename` is the bare name given to
325     /api/compile
326     (which only writes into output_dir; the browser previously got back an unreachable
327     server-local path)."""
328     try:
329         name = safe_output_filename(filename)
330     except ValueError as e:
331         raise HTTPException(status_code=400, detail=str(e)) from e
332     if not db_instance.get_video(video_id):
333         raise HTTPException(status_code=404, detail="Indexed video record not found.")
334     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
335     or "output"
336     path = os.path.join(output_dir, name)
337     try:

```

```

335         path = resolve_under_root(path, output_dir)
336     except ValueError as e:
337         raise HTTPException(status_code=400, detail=str(e)) from e
338     if not os.path.isfile(path):
339         raise HTTPException(status_code=404,
340                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
341     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
342     return FileResponse(path, media_type=media, filename=name)
343
344
345     @app.post("/api/query")
346     def query_play_segments(req: QueryRequest):
347         filters = {
348             "server": req.server,
349             "receiver": req.receiver,

```

50. assistant (2026-06-21T02:35:40.572Z)

Perfect! I now have a comprehensive understanding of the codebase. Let me create the structured output with my findings:

51. user (2026-06-21T02:36:19.925Z)

Structured output provided successfully

52. assistant (2026-06-21T02:36:28.392Z)

Summary

I've mapped the deployment profile seams that orchestrate INPUT-SOURCE + STORAGE + COMPUTE backend selection. Here are the key findings:

****TODAY'S STATE:****

****Input Source:**** Hardcoded to local filesystem. `ProcessRequest.video_path` is a plain string (always local). The worker supports URIs (``s3://``, ``gcs://``, ``local://``, ``gdrive://``) via ``storage.fetch()``, but the main CLI has no input abstraction.

****Compute Backend:**** Semi-exposed via environment variable ``RALLY_DETECTOR_IMPL`` or config ``indexing.detector_impl`` (``auto`=WSL`, ``native`=GPU-box Linux`, ``stub`=CPU mock`), but only trajectory hybrids use it. No top-level orchestration picks GPU vs Cloud-Run coherently.

****Storage:**** Fully modular. ``backend/storage/`` supports 4 backends (local/s3/gcs/gdrive) via URI parsing. Config knobs exist: ``storage.backend``, ``storage.output_root``, ``storage.workspace_root``, ``storage.workspace_prefix``, and the per-video workspace helper (``backend/utils/workspace.py``) is ready but gated by ``single_folder_per_video=False``.

****AI Handoff:**** Controlled by ``indexing.skip_ai_handoff`` boolean (local-noai mode when True; cloud Gemini path when False).

****NET-NEW GAPS:****

1. ****Input source selection**** ? needs an ``input_backend`` + ``input_root`` parallel to output-side ``StorageConfig``
2. ****Deployment profile**** ? no preset bundles or auto-detection of environment (Cloud Run vs local GPU box vs CI)
3. ****Startup environment checks**** ? no detection of `K_SERVICE` (Cloud Run), `torch.cuda` (GPU), or CI context
4. ****Per-video workspace convergence**** ? helper exists, but write sites not yet migrated (gated by ``single_folder_per_video`` flag, default OFF per `PER_VIDEO_WORKSPACE.md` phases P2?P4)

****REUSE OPPORTUNITIES:****

The existing ``storage.*`` config knobs + ``detector_impl`` + ``skip_ai_handoff`` can already build coherent profiles when composed. The worker is already profile-agnostic and accepts ``--config`` + ``--set k=v`` overrides. No core rework needed? just add a ``deployment`` config section (profile name) + env-var-driven startup logic, and expose input-source selection to the main CLI.